

Fxxk-CG

Fxxk CG 考试内容

21 学期考试内容:

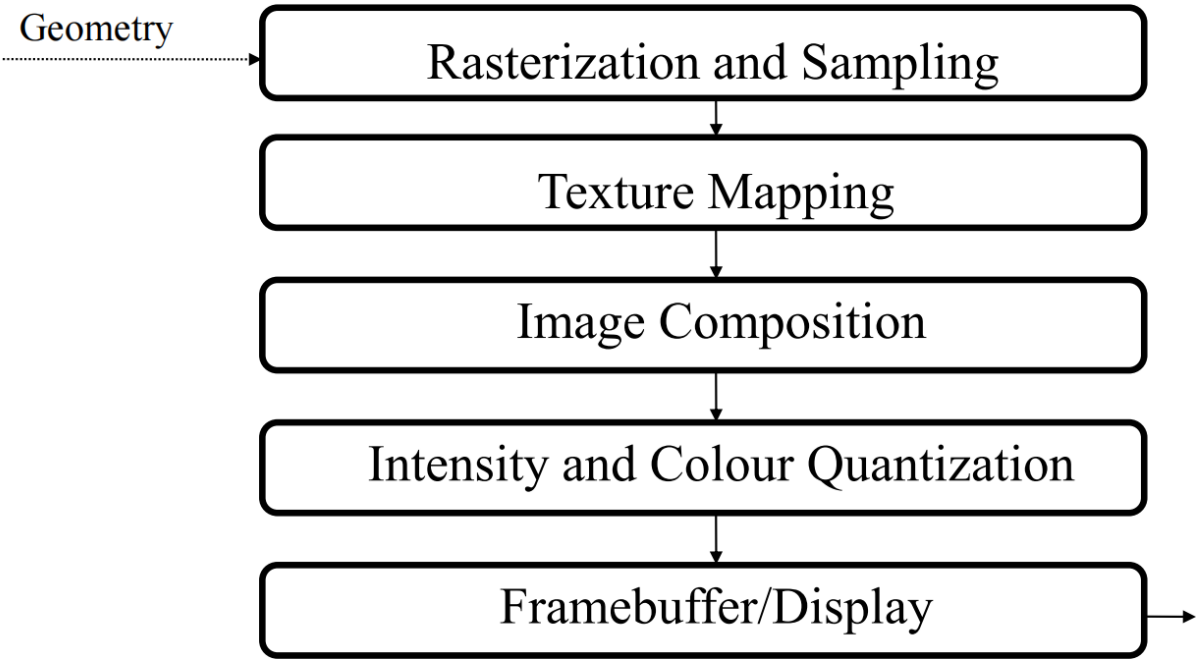
- 大部分内容考察知识点的理解, 比如用自己的话阐释两个概念之间的区别
- 一题考察 OpenGL 程序的简单填空 (写了作业就会做)
- 一题考察了 hermite 样条的推导, 在 ppt 上有详细的过程。

22 学期考试内容:

- 部分考题来自于平时的 Assignment
- 考试范围不超过老师考前提供的 Review
- Curves 在 Review 中只出现了一行, 但是考了一个问答题 (4 分) 和两道大题 (各 8~9 分, 具体分数记不清楚) 占比蛮大
- 考题全为问答题的形式, 其中分值较大的题涉及边坍塌、用 Half-edge 结构遍历相邻的点的伪代码。

1 Introduction

Imaging Pipeline



Geometry (几何)

Rasterization and Sampling (光栅化和采样)

Texture Mapping (纹理映射)

Image Composition (图像合成)

Intensity and Colour Quantization (强度和颜色量化)

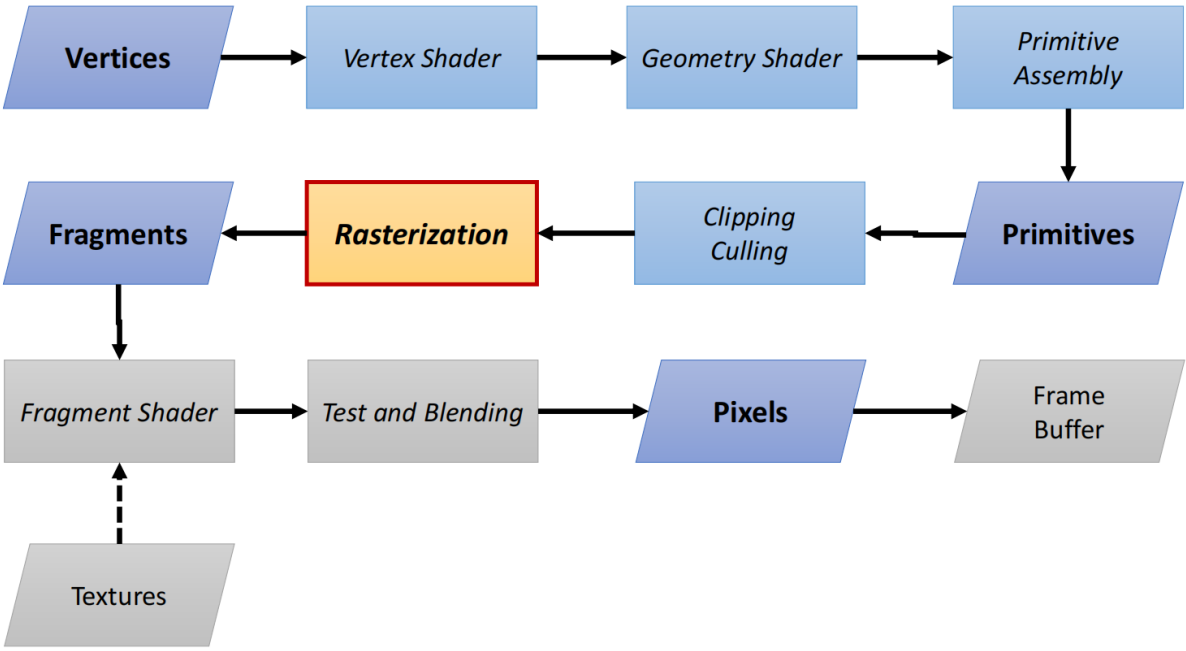
Framebuffer/Display (帧缓冲/显示)

| [图片]



赵嘉禾 12月24日 21:35
尝试评论

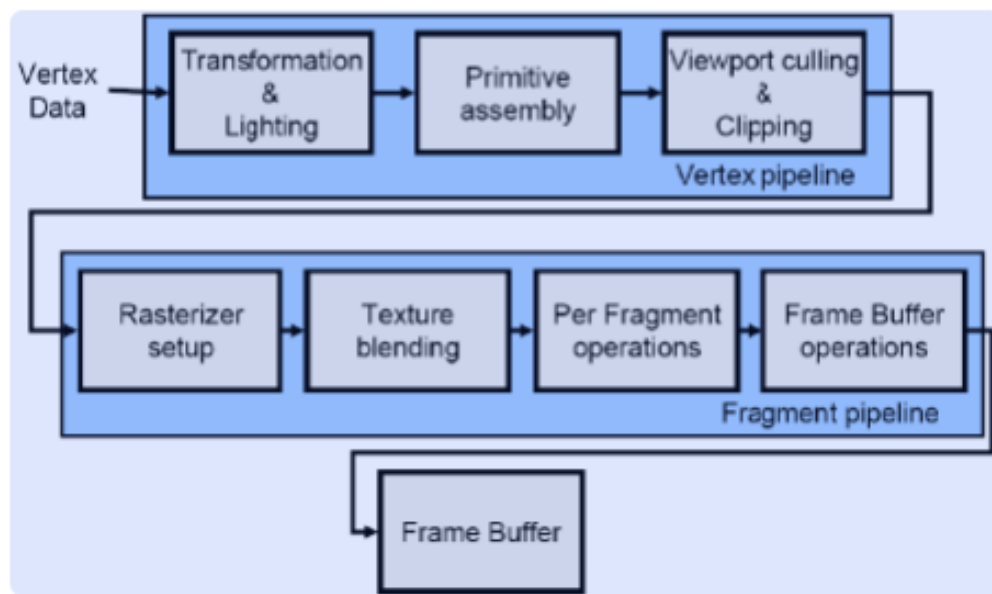
The graphics pipeline



这张图展示了图形管线（graphics pipeline）的流程：

1. **Vertices（顶点）**：这是图形管线的起点，顶点是构成几何图形的基本元素，通常包含位置、颜色、纹理坐标等信息。
2. **Vertex Shader（顶点着色器）**：顶点着色器处理每个顶点的的数据，可以进行变换、光照、纹理坐标变换等操作，为后续的几何处理准备数据。
3. **Geometry Shader（几何着色器）**：几何着色器处理顶点集合，可以生成新的顶点、改变几何图形的拓扑结构，或者进行其他几何变换。
4. **Primitive Assembly（图元组装）**：这个阶段将顶点组装成图元（如点、线、三角形），为光栅化做准备。
5. **Clipping Culling（裁剪剔除）**：在这个阶段，系统会剔除视图之外的图元，以及在视口内部进行裁剪，以减少渲染的计算量。
6. **Primitives（图元）**：经过裁剪和剔除后，剩余的图元将被送入光栅化阶段。
7. **Rasterization（光栅化）**：光栅化是将图元转换为像素的过程。这个阶段会确定哪些像素属于当前图元，并为这些像素生成片段（fragments）。
8. **Fragments（片段）**：片段是光栅化后的结果，每个片段对应一个像素，包含颜色、深度等信息。
9. **Fragment Shader（片段着色器）**：片段着色器处理每个片段的数据，可以进行纹理采样、光照计算、融合等操作，最终确定像素的颜色值。
10. **Test and Blending（测试和混合）**：这个阶段进行深度测试、模板测试等，以确定哪些片段应该被显示。混合（blending）则用于处理透明效果。
11. **Pixels（像素）**：经过测试和混合后，最终的像素数据被写入帧缓冲区。
12. **Frame Buffer（帧缓冲区）**：帧缓冲区存储最终的像素数据，最终这些数据会被显示在屏幕上。
13. **Textures（纹理）**：纹理是用于给几何图形添加细节的图像。在片段着色器中，纹理坐标会被用来从纹理中采样颜色值，以增加视觉真实感。

OpenGL



Programmable: Allow programmer and artist to define the vertex and fragment (and geometry) processing

3 What are vertex, primitive and fragment?

- A vertex is a point/node in the space, represented by its coordinate and properties, e.g. color, normal, texture coordinate etc.

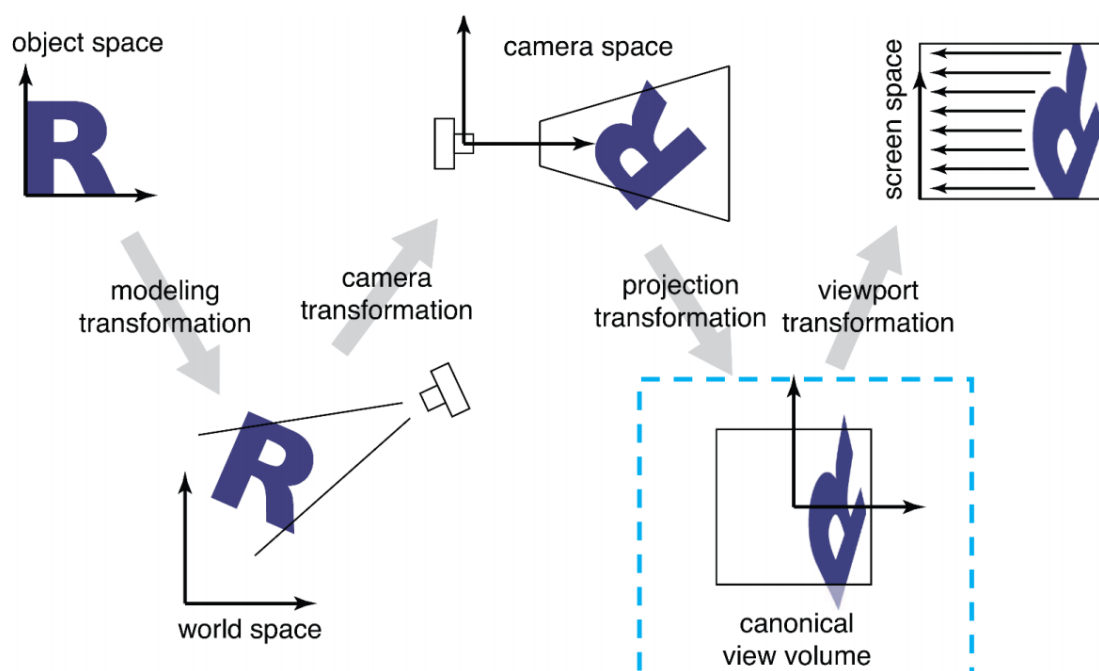
A primitive is the basic element of geometric shapes and rendering, it is composed of vertices. It can be points, lines, polygons or polyhedrons.

A fragment is the output data of the geometry shader in the rasterization stage. The fragment contains the data necessary to generate a single pixel in the frame buffer, e.g. color, depth, normal, texture coordinate etc.

2 Viewing and Modeling Basics

view transformation

Viewing Transformations



视图变换的四个主要阶段

1. Modeling Transformation

将物体的局部坐标系变换到世界坐标系。

2. Camera Transformation (View Transformation)

将世界坐标系变换到观察坐标系（或称视图坐标系）。

3. Projection Transformation

将观察坐标系中的三维坐标投影到裁剪空间（Clip Space），然后通过齐次坐标除法（即将点的 x,y,z 除以 w ）将坐标归一化为 NDC。

4. Viewport Transformation

将 NDC 转化为屏幕坐标（Screen Coordinates），根据视口的尺寸映射到屏幕像素坐标。

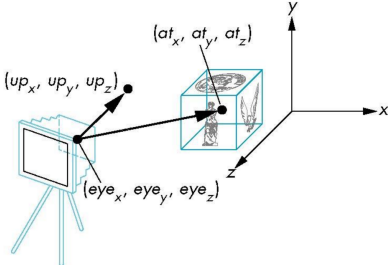
Virtual Camera

How to model a virtual camera?

- pose
- focal length
- size of film
- Which can be used for zooming?

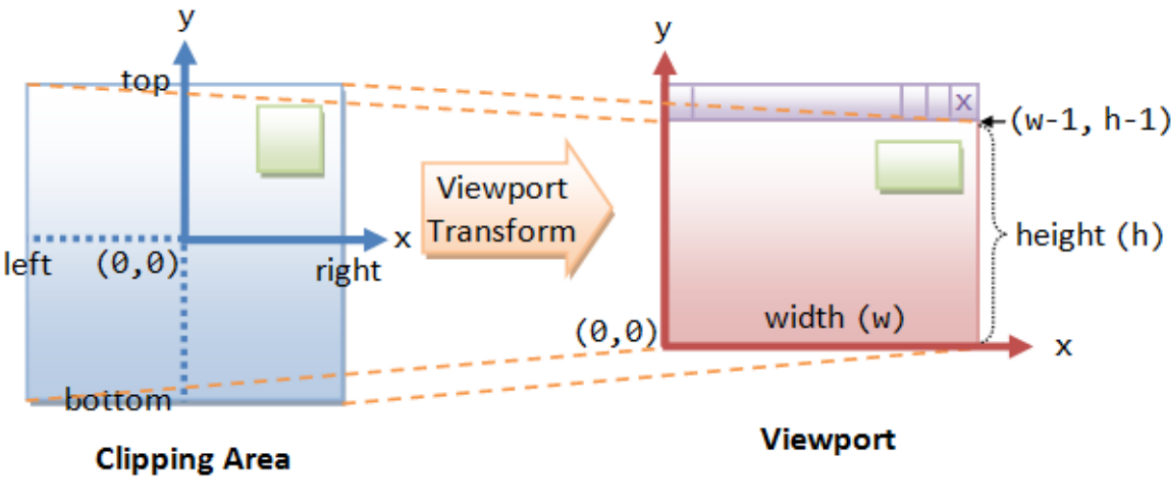
gluLookAt

```
gluLookAt(eyex, eyey, eyez, atx, aty, atz, upx, upy, upz)
```



- 1. at（目标点）
 - 含义：
 - at 代表观察的目标点（target point）。它是一个三维坐标，表示摄像机所指向的位置。
 - 在图形学中，摄像机（或观察点）会朝着这个目标点进行观察，决定了场景中的哪个部分会出现在摄像机的视野中心。
 - 图中的表示：
 - 在图中，(at_x, at_y, at_z) 标识了目标点的位置。这个点是摄像机视线的终点。
- 2. up（上方向向量）
 - 含义：
 - up 代表上方向向量（up vector）。它也是一个三维向量，用于定义摄像机的“上”方向。
 - 这个向量决定了摄像机如何倾斜。例如，在默认情况下，上方向向量通常是 (0, 1, 0)，表示摄像机的上方向是沿着 y 轴正方向。但可以通过改变这个向量来实现倾斜或旋转摄像机的效果。

The relationship of Clipping Window and Viewport



Orthographic（正交投影）

Frustum（视锥）

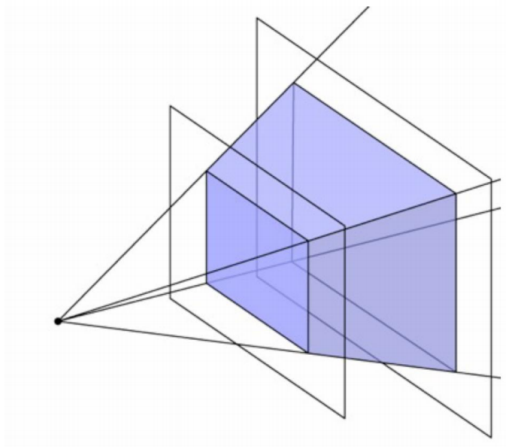
Orthographic（正交投影）



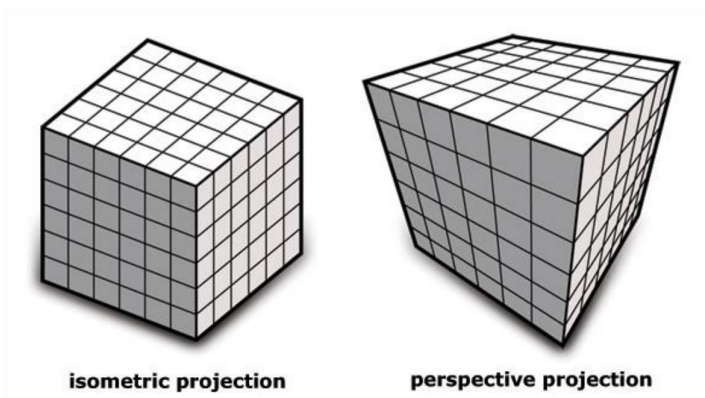
赵嘉禾 12月24日 22:40

Projection（透视投影）：
将三维物体投影到二维平面上，三个坐标 (x, y, z) 与视平面距离有关。
每个点沿着摄像机视线方向投影到视平面上，使得三维物体可以以二维形式呈现在屏幕上。
Orthographic（正交投影）：
将三维物体投影到二维平面上，不考虑物体与视平面的距离，所有物体都按照相同的比例投影到屏幕上。
正交、透视、斜交等方向。
用于表示物体的精确比例和形状。

a 3D clipping volume called **frustum**



Perspective projection



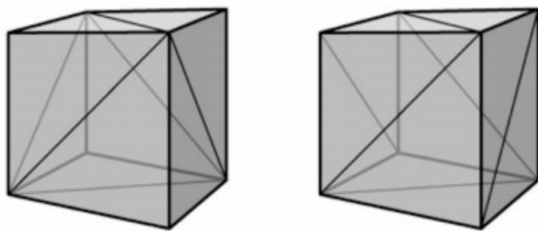
Models

- The data describing the size, position, and shape of (3D) objects (ISO10303 Abbrev.)

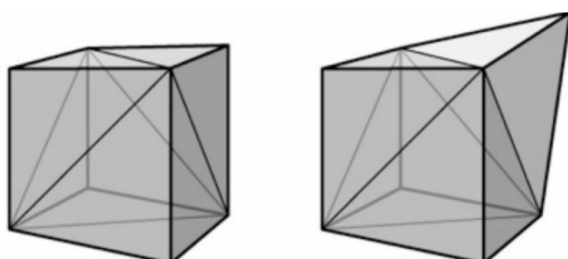
Geometry (几何) 与 Topology (拓扑)

Geometry vs Topology

- Same geometry, different topology



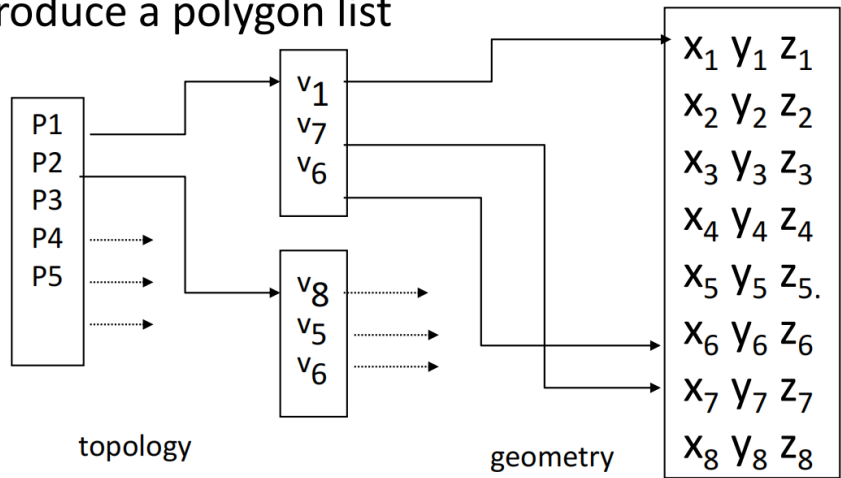
- Same topology, different geometry



PS. Geometry means the locations of the vertices, while Topology means the organization of vertices and edges. Thus Topology holds even Geometry changes.

Polygon指向一组vertices指向具体的坐标

Introduce a polygon list



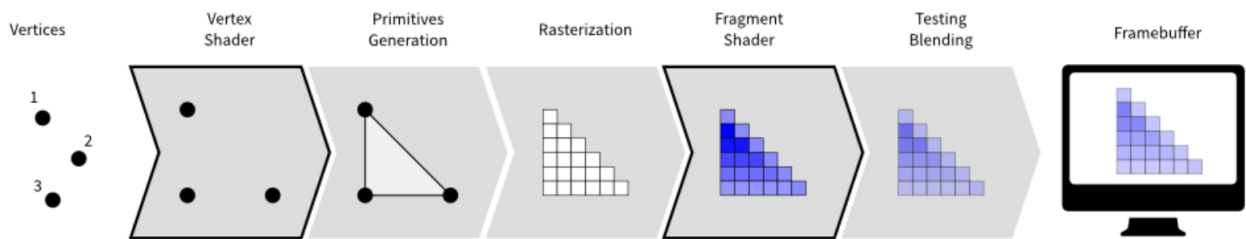
Cons and Pros of Polygonal mesh 多边形网格

- **Pros:** flexible and computers can render them quickly. The vast majority of 3D models today are built as textured polygonal models
- **Cons:** polygons are planar and need approximate curved surfaces using many polygons, representation is not unique



除此之外的模型方法：Constructive solid geometry (CSG)（建构实体几何）、Volume-based representation（基于体积的建模）、Point based representation（基于点云的建模）

Opengl rendering pipeline



Affine transformation（仿射变换）

- Translation together with linear transformations - Affine transformation

在Cartesian coordinate（笛卡尔坐标系）中，变换只需要2x2的矩阵，但为了平移、缩放、旋转以及仿射变换的统一处理，后续使用Homogeneous Coordinates（齐次坐标）的3x3变换矩阵。

Affine transformation（仿射变换）



赵嘉禾 12月24日 23:34
对仿射变换理解：面对公式 $P'=P+d$ ，可以使用矩阵相乘的办法直接得到运算，而不必一步一步算。

convert w
 x, y, z
side by it to get Cartesian coordina
[1]
 $y, z, 1$
 x', y', z'
 $-dx, -dy, -dz$

note that this expres
Three dimensions a
point = point + vect
Point - point = ?
Can we translating

now express translation using
atrix T_d in homogeneous coord
s, where
 $T_d = T_d(dx, dy) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$

$$\begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + dx \\ y + dy \\ 1 \end{bmatrix}$$



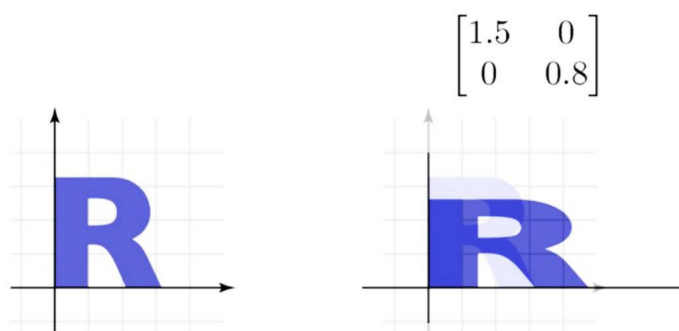
Hasky 12月24日 23:35
yes

- Compose translation and linear transformations using homogenous coordinates
- $$\begin{bmatrix} M_S & d_S \\ 0 & 1 \end{bmatrix} \begin{bmatrix} M_T & d_T \\ 0 & 1 \end{bmatrix} \begin{bmatrix} P \\ 1 \end{bmatrix} = \begin{bmatrix} M_S M_T P + M_S d_T + d_S \\ 1 \end{bmatrix}$$
- This form is better for implementation because all **Affine Transformations** can be expressed this way and multiple transformations can be concatenated together

缩放

Non-uniform Scale

$$\bullet \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} s_x \times x \\ s_y \times y \end{bmatrix}$$



平移

$$T_d = T_d(dx, dy) = \begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + dx \\ y + dy \\ 1 \end{bmatrix}$$

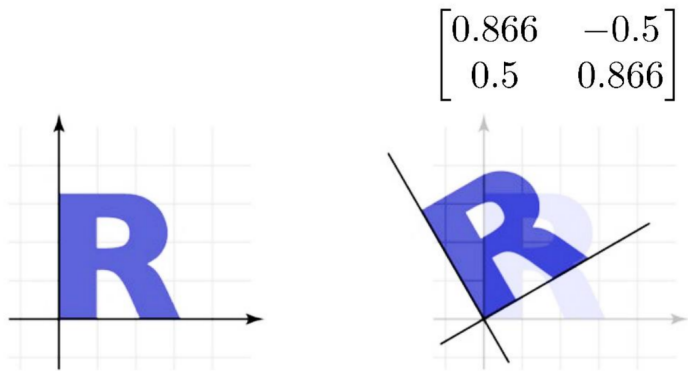
旋转

$$R = R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x\cos\theta - y\sin\theta \\ x\sin\theta + y\cos\theta \\ 1 \end{bmatrix}$$

Rotation

• $\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x\cos\theta - y\sin\theta \\ x\sin\theta + y\cos\theta \end{bmatrix}$

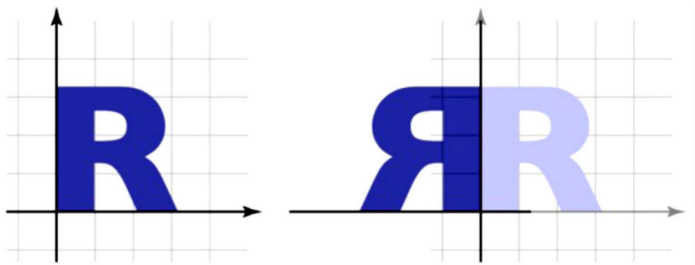


翻转

Reflection

- Can be considered as a special case of non-uniform scale

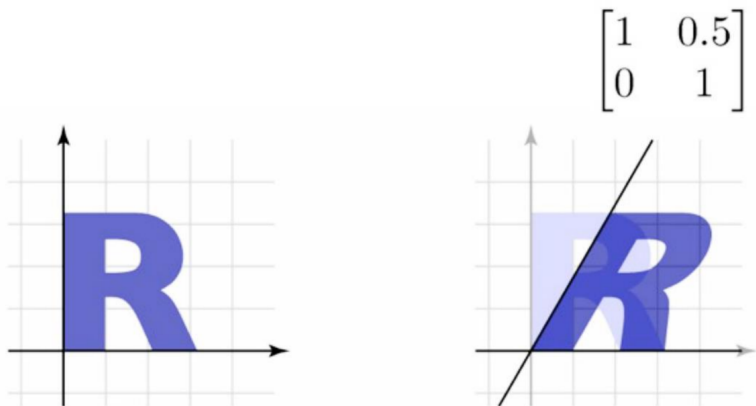
• $\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$



Shear

Shear

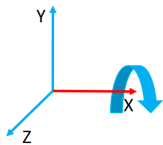
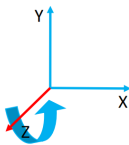
• $\begin{bmatrix} 1 & \alpha \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + \alpha y \\ y \end{bmatrix}$



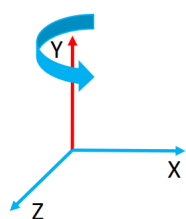
(三维)

• $R_z(\theta) = \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$

• $R_x(\theta) = \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$



$$\bullet R_y(\theta) = \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

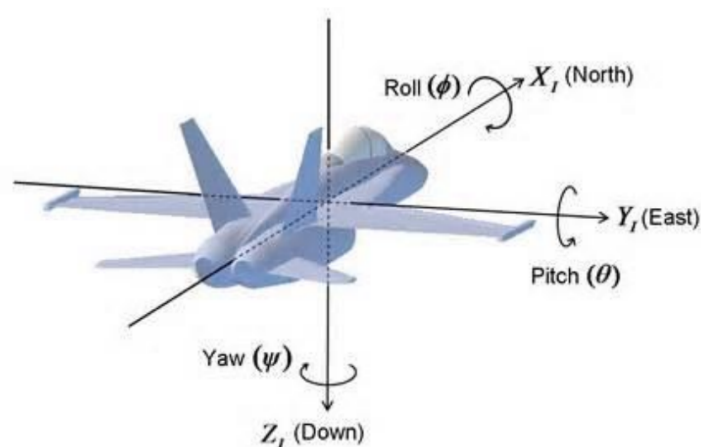


Rotation Presentation 旋转表示方法

Euler Angles (欧拉角) : 可能发生Gimbal Lock (万向锁) :

当使用欧拉角表示旋转时，如果两个旋转轴在某一特定姿态下对齐或接近共面，就会导致旋转自由度丢失。这种现象被称为 **Gimbal Lock (万向锁)**。

- Roll(Φ) around X
- Pitch(Θ) around Y
- Yaw(Ψ) around Z



Quaternions (四元组)

它与常用的另外两种表示方式（三维正交矩阵和欧拉角）是等价的，但是避免了欧拉角表示法中的万向锁问题。比起三维正交矩阵表示，四元数表示能够更方便地给出旋转的转轴与旋转角。

用一位实数和三维虚数（i、j、k为互相垂直的单位虚数）表示。

Representation

- A quaternion is composed of one real element and three complex elements

$$q = q_0 + q_1 i + q_2 j + q_3 k$$

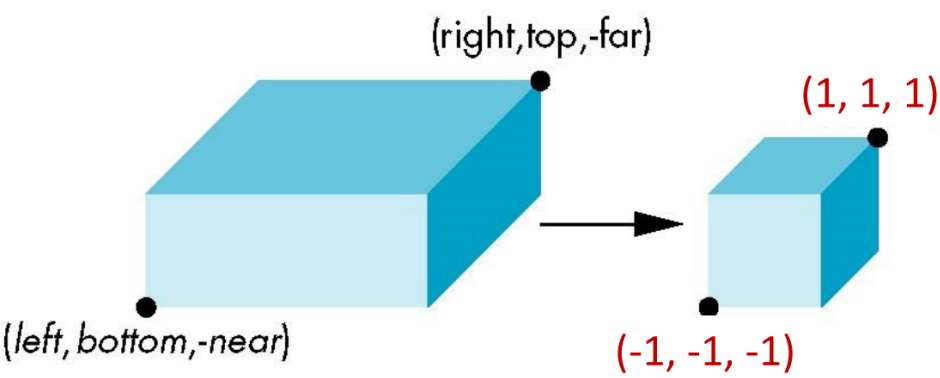
$$\begin{cases} i^2 = j^2 = k^2 = -1 \\ ij = k, ji = -k \\ jk = i, kj = -i \\ ki = j, ik = -j \end{cases}$$

A rotation through an angle of θ around the axis defined by a unit vector $n = [n_x, n_y, n_z]^T$ can be represented by a quaternion:

$$q = \left[\cos \frac{\theta}{2}, n_x \sin \frac{\theta}{2}, n_y \sin \frac{\theta}{2}, n_z \sin \frac{\theta}{2} \right]^T$$

Projection Method 投影方法

Orthogonal Projection (正交投影)



Orthogonal Matrix 正交投影矩阵

Two steps

- Move center to origin
 $T(-(left+right)/2, -(bottom+top)/2, (near+far)/2))$
- Scale to have sides of length 2
 $S(2/(right-left), 2/(top-bottom), 2/(-far -(- near)))$

$$P = ST = \begin{bmatrix} \frac{2}{right - left} & 0 & 0 & -\frac{right + left}{right - left} \\ 0 & \frac{2}{top - bottom} & 0 & -\frac{top + bottom}{top - bottom} \\ 0 & 0 & \frac{2}{near - far} & -\frac{far + near}{far - near} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

随后，让 $z=0$

- Set $z = 0$
- Equivalent to the homogeneous coordinate transformation

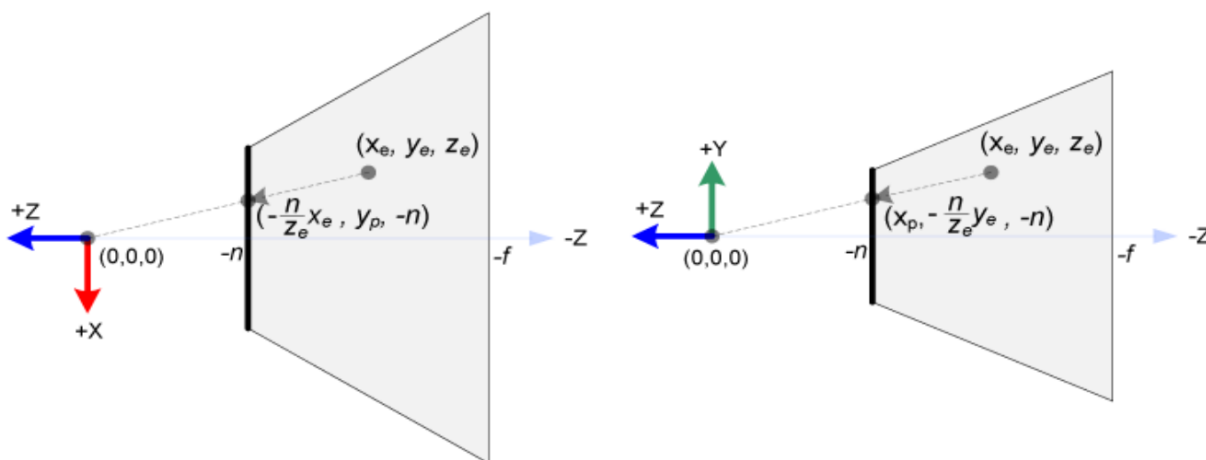
$$\mathbf{M}_{\text{orth}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Hence, general orthogonal projection in 4D is $\mathbf{P} = \mathbf{M}_{\text{orth}} \mathbf{S} \mathbf{T}$

Perspective Projection 透视投影

1. X_e 到 X_p 的转化 (透视投影)

- $x_p = -\frac{n}{z_e} x_e$
- $y_p = -\frac{n}{z_e} y_e$



2. X_e 到 X_n 的转化 (标准化)

- $x_n = \frac{2}{r-l} \cdot x_p - \frac{r+l}{r-l}$
- $y_n = \frac{2}{t-b} \cdot y_p - \frac{t+b}{t-b}$
- Substitute x_e and y_e into the equations
- $x_n = (\frac{2n}{r-l} \cdot x_e + \frac{r+l}{r-l} \cdot z_e) / -z_e$
- $y_n = (\frac{2n}{t-b} \cdot y_e + \frac{t+b}{t-b} \cdot z_e) / -z_e$

3. 中间变量 X_c (X-Clipping, 是 $-Z_e$ 倍的 X_n)

问号的一行通过设未知数解方程得到 (见笔记蓝字部分)

$$\bullet \begin{pmatrix} x_c \\ y_c \\ z_c \\ w_c \end{pmatrix} = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ ? & ? & ? & ? \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} x_e \\ y_e \\ z_e \\ 1 \end{pmatrix}$$

$$\bullet \begin{pmatrix} x_n \\ y_n \\ z_n \\ 1 \end{pmatrix} = \begin{pmatrix} x_c/w_c \\ y_c/w_c \\ z_c/w_c \\ 1 \end{pmatrix}$$

完整过程，设 (x_e, y_e, z_e) 为原始点的坐标，投影到的平面为 $Z=n$

(注意蓝字部分的 n 是 near，近的一处的 z 值，变量名设计有歧义是大不牛ppt写的不好导致的。。。)

plane: $z=n$ 透視

$$\begin{cases} x_p = -\frac{n}{z_e} \cdot x_e \\ y_p = -\frac{n}{z_e} \cdot y_e \end{cases} \quad \begin{cases} x_n = \frac{z}{r-l} x_p - \frac{r+l}{r-l} \\ y_p = \frac{z}{t-b} y_p - \frac{t+b}{t-b} \end{cases} \quad \text{Normalize}$$

$$\Rightarrow \begin{cases} x_n = \left(\frac{2n}{r-l} x_e + \frac{r+l}{r-l} \cdot z_e \right) / -z_e \\ y_n = \left(\frac{2n}{t-b} y_e + \frac{t+b}{t-b} \cdot z_e \right) / -z_e \end{cases}$$

$$\therefore \begin{pmatrix} x_c \\ y_c \\ z_c \\ w_c \end{pmatrix} = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & 2A & B \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} x_e \\ y_e \\ z_e \\ 1 \end{pmatrix}$$

$w_c = -z_e$

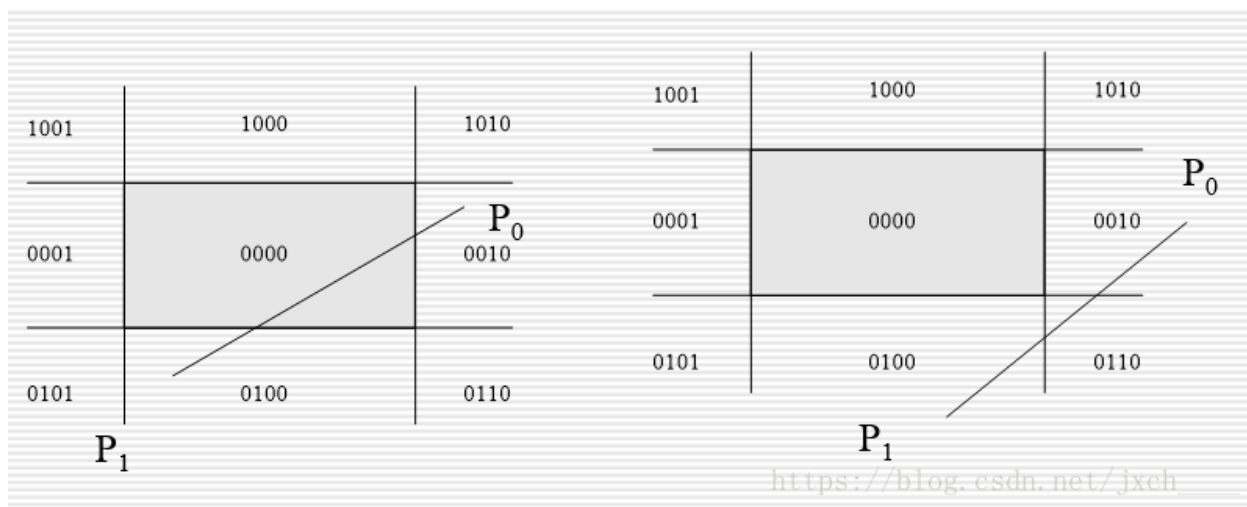
$$\begin{pmatrix} x_n \\ y_n \\ z_n \\ 1 \end{pmatrix} = \begin{pmatrix} x_c \\ y_c \\ z_c \\ w_c \end{pmatrix} / -z_e$$

$$\therefore \begin{pmatrix} x_c \\ y_c \\ z_c \\ w_c \end{pmatrix} = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{n+f}{n-f} & \frac{2nf}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} x_e \\ y_e \\ z_e \\ 1 \end{pmatrix}$$

$z_c = A z_e + B = -z_e \cdot z_n$
 $(-n, -f) \rightarrow (-1, 1)$
 $\begin{cases} -n = -nA + B \\ f = -fA + B \end{cases}$
 $\Rightarrow \begin{cases} A = \frac{n+f}{n-f} \\ B = \frac{2nf}{n-f} \end{cases}$

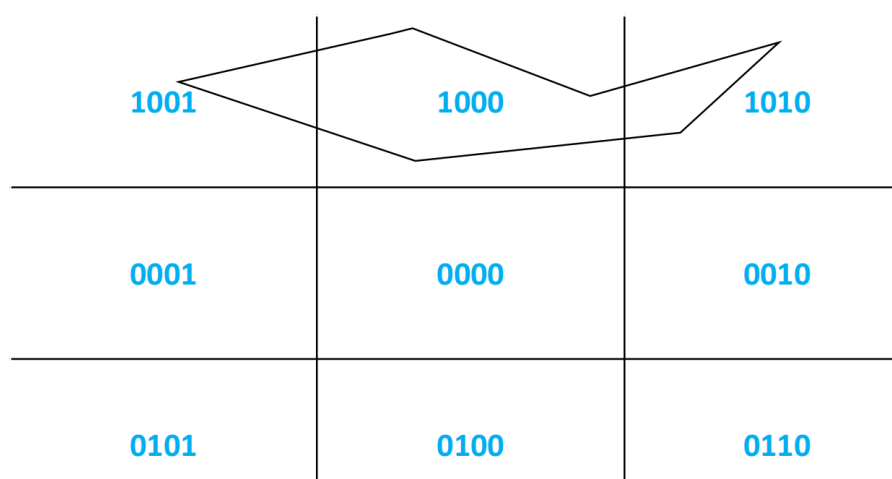
Clipping Algorithm

Clipping 2D Line Segments (划分线算法) : Cohen-Sutherland Algorithm



Using Outcodes

- It works for arbitrary primitives
- And for arbitrary dimensions
- $1001 \& 1000 \& 1010 \& 1010 \& 1000 \& 1000 = 1000$
Reject!



把所有点逻辑AND，得到如果不是0，那就拒绝。

如果是0，那就要进一步判断各个点是不是有不同的1，如果有的话缩短线段再进行如此的判断。

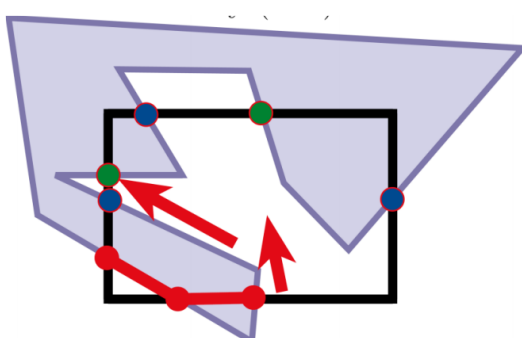
Clipping 2D Polygon（划分多边形算法）：Weiler-Atherton Clipping

WALK Rules

In-to-out 走Polygon

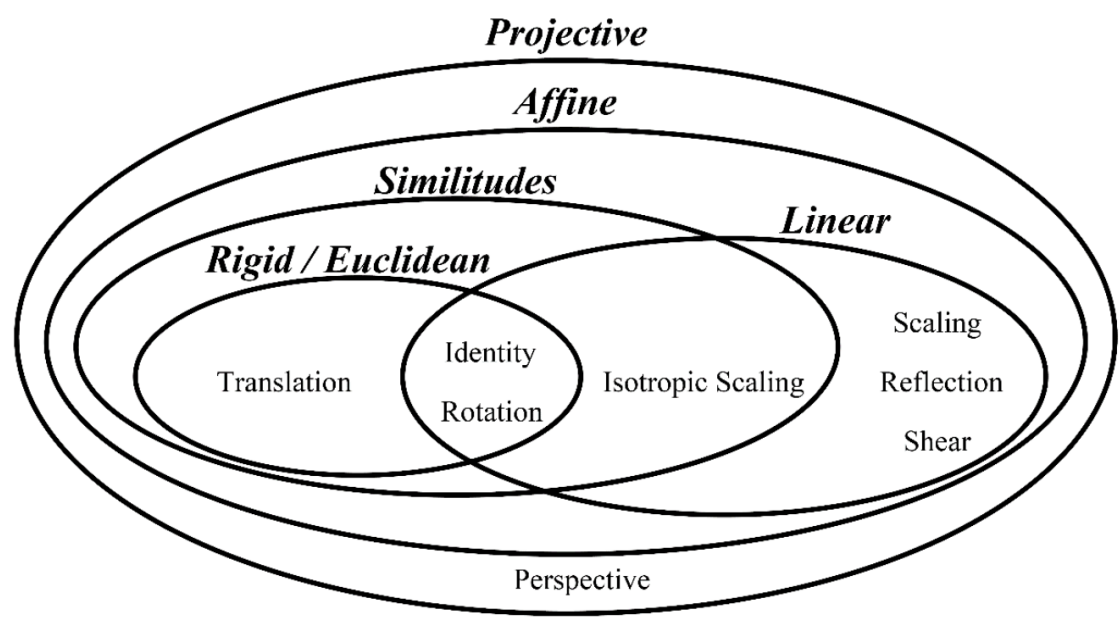
Out-to-in 走window边框

- Out-to-in point:
 - Record clipped point
 - Follow polygon boundary (ccw)
- In-to-out point:
 - Record clipped point
 - Follow window boundary (ccw)



Cons: Problem with floating point precision

Transformation的关系



1. **Identity (恒等变换)**：这是所有变换的核心，它不改变任何东西。
2. **Rotation (旋转)** 和 **Translation (平移)**：这两种变换是刚体变换 (Rigid transformations) 的一部分，它们保持图形的形状和大小不变。
3. **Rigid / Euclidean (刚体/欧几里得变换)**：包括旋转、平移和恒等变换。这些变换保持图形的形状、大小和角度不变。
4. **Isotropic Scaling (各向同性缩放)**：在所有方向上以相同的比例缩放，因此也属于刚体变换。
5. **Similarities (相似变换)**：包括刚体变换和各向同性缩放。这些变换保持图形的形状不变。
6. **Affine (仿射变换)**：包括相似变换和剪切 (Shear)。仿射变换保持平行线和比例，但不一定保持角度和距离。
7. **Linear (线性变换)**：包括缩放、反射和剪切。线性变换是仿射变换的一部分，它们可以通过矩阵乘法来表示。
8. **Perspective (透视变换)**：这种变换不保持平行线，它可以模拟三维空间中的视角效果。
9. **Projective (投影变换)**：这是最广泛的一类变换，包括透视变换。

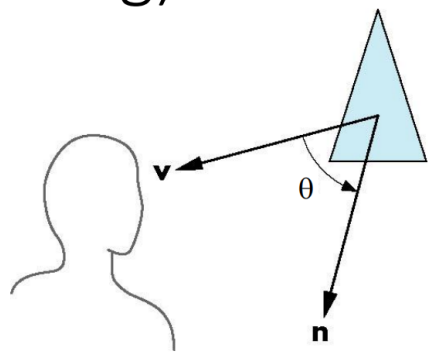
Visible surface algorithms 可见表面算法

Object space techniques (对象空间技术)：在光栅化前完成

1. Background-Face Removal

Back-Face Removal (Culling)

- Face is visible iff $90 \geq \theta \geq -90$
- Equivalently $\cos \theta \geq 0$ or $\mathbf{v} \cdot \mathbf{n} \geq 0$
- In eye coordinate frame need only test the sign of the Z component

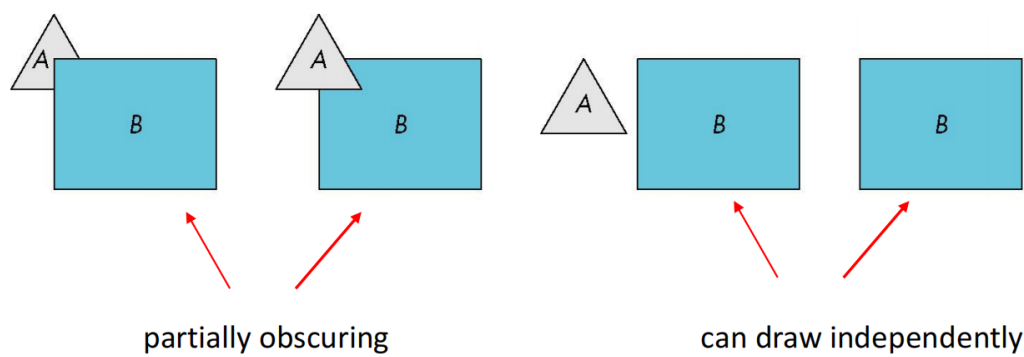


2. Hidden Surface Remval

Cons: High complexity for comparing every two objects

Hidden Surface Removal

- Object-space approach: use pairwise testing between polygons (objects)

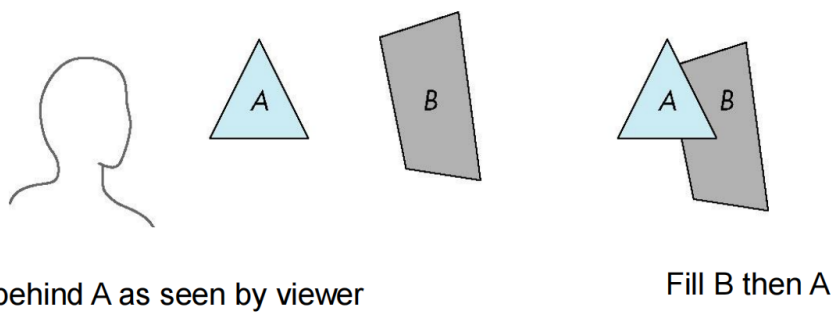


3. Painter's Algorithm

先进行时间复杂度 $O(n \log n)$ 的物体排序

Painter's Algorithm

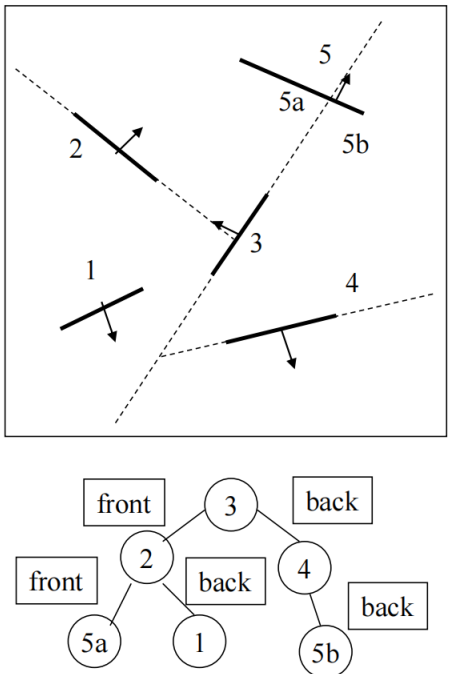
- Render polygons a back to front order so that polygons behind others are simply painted over



4. BSP (Binary Space Partitioning) Tree

BSP Tree

- Choose polygon arbitrarily
- Divide scene into front (relative to normal) and back half-spaces.
- Split any polygon lying on both sides.
- Choose a polygon from each side – split scene again.
- **Recursively divide each side until each node contains only 1 polygon.**



[图片]



赵嘉禾 12月25日 14:44

按照得到的树，根据 viewer 位置，依次递归渲染另一侧树、根节点、同一侧树



Hasky 12月25日 14:50

Selon l'arbre obtenu, selon la position du spectateur, la récursivité rend l'autre arbre latéral, le nœud racine et le même arbre latéral en séquence



Hasky 12月25日 14:51

Entsprechend dem erhaltenen Baum, entsprechend der Betrachterposition, rendert die Rekursion den anderen Seitenbaum, den Wurzelknoten und denselben Seitenbaum nacheinander



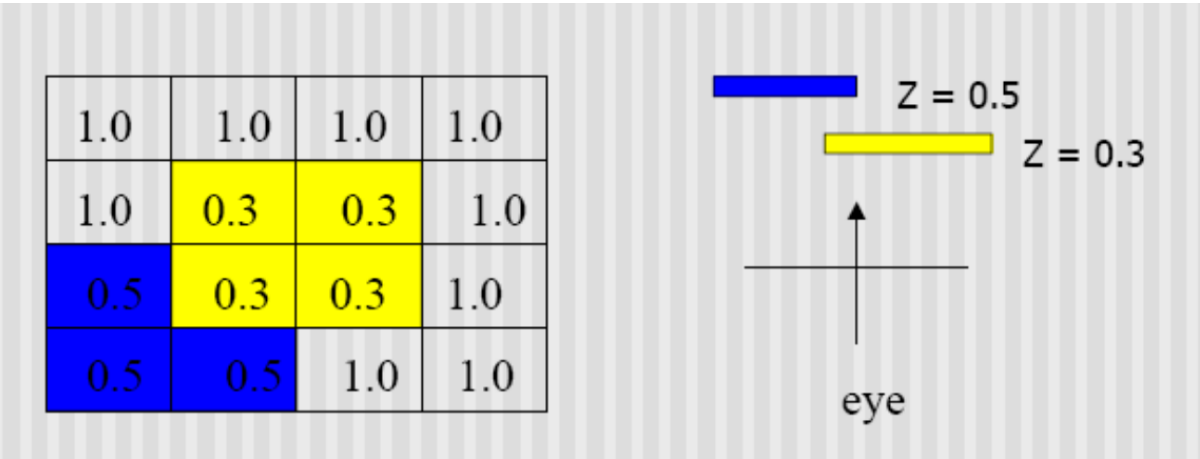
赵嘉禾 12月25日 14:51

Start at root polygon.

- If viewer is in front half-space, draw polygons behind root first, then the root polygon, then polygons in front.
- If viewer is in back half-space, draw polygons in front of root first, then the root polygon, then polygons in the back.
- If polygon is on edge – either can be used.
- Recursively descend the tree.

Image space techniques（图像空间技术）：光栅化时完成

1. Z-buffer



Advantage

- Simple to implement in hardware.
 - Memory for z-buffer is now not expensive
- Diversity of primitives – not just polygons.
- Unlimited scene complexity
- Don't need to calculate object-object intersections.

Disadvantage

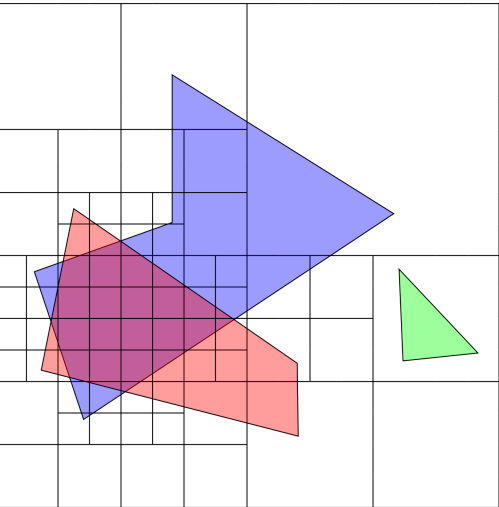
- Extra memory and bandwidth
- Waste time drawing hidden objects

Z-precision errors

- May have to use point sampling

Z-buffer可与object-based方法结合，如：

1. Area Subdivision (Warnock's Algorithm)



2. Portal Culling



@Hasky 螳臂



赵嘉禾 12月25日 14:52

@赵嘉禾 BSP tree 是不是应该认为是一种实现 painter 算法的办法



赵嘉禾 12月25日 14:52

根据 review

ie 2 Viewing

ipping in clip space rather than in
ing in Viewing frustum?
ing in NDC?
g algorithms
nes
polygons
it's algorithm VS Z-buffering
in order



Hasky 12月25日 15:04

@赵嘉禾 应该可以算

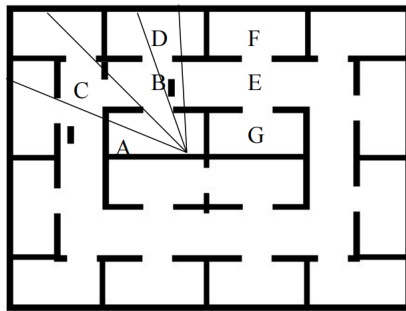
Portal Culling (object-space)

Model scene as a graph:

- Nodes: Cells (or rooms)
- Edges: Portals (or doors)

Graph gives us:

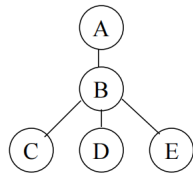
- Potentially visible set



1.Render the room

2.If portal to the next room is visible, render the connected room in the portal region

3.Repeat the process along the scene graph



3 Rendering and Rasterization

Vector Graphics (矢量图形) 、 Raster Graphics (光栅图形)

interpolate (内插)

Rasterization 光栅化

Transform continuous primitive into discrete

First job: enumerate the pixels covered by a primitive

- simple, aliased definition: pixels whose centers fall inside

Second job: interpolate values across the primitive

- e.g. colors computed at vertices
- e.g. normals at vertices

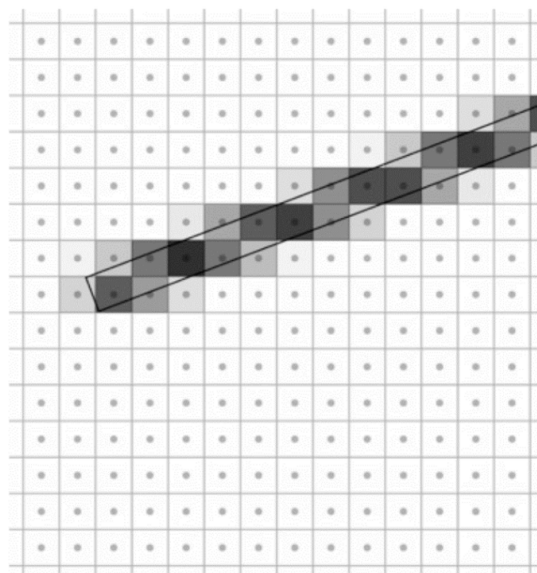
Problem: Brightness varies with slope.

Method: Antialiasing (抗锯齿)

Eg. Box filtering、 Weighted filtering、 Xiaolin Wu's Algorithm

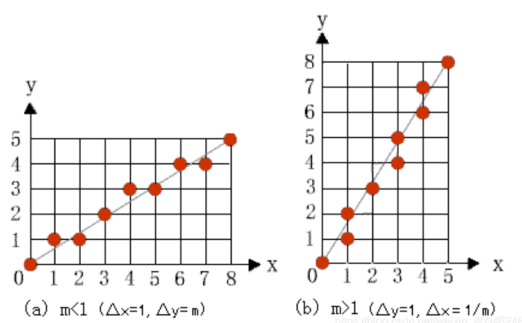
Basic idea: replace “is the image black at the pixel center?” with “how much is pixel covered by black?”

Replace yes/no question with quantitative question.



(2013 Steve Marschner)

DDA 算法



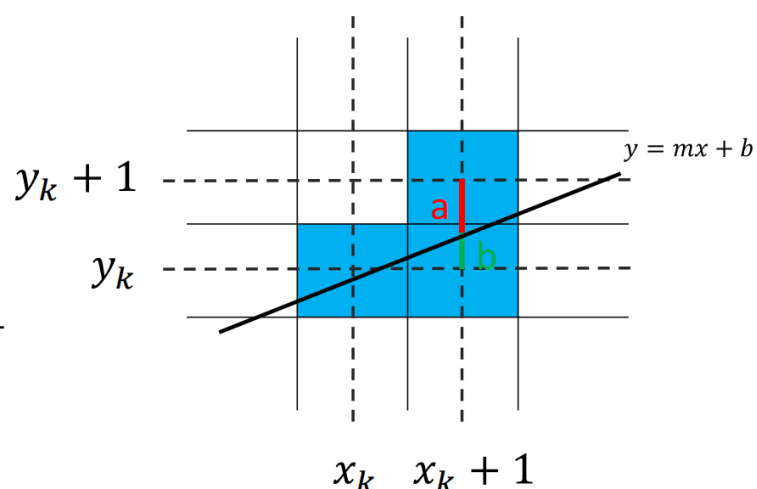
```
dx = x2-x1; dy = y2-y1;  
If |dx| > |dy| then step = dx else step = dy;  
For(i=0; i<step,i++) {  
    x+=dx/step;  
    y+=dy/step;  
    SetPixel(round(x), round(y));  
}
```

The short comings of DDA:

- Floating point arithmetic
- Round operation

Bresenham's Algorithm

- If $a < b$
 - Then upper pixel
- If $a > b$
 - Then lower pixel
- $a = (y_k + 1) - y = y_k + 1 - m(x_k + 1) - b$
- $b = y - y_k = m(x_k + 1) + b - y_k$
- let $d = \Delta x(b - a) = 2\Delta y \cdot x_k - 2\Delta x \cdot y_k + c$



d is an integer
d > 0 use upper pixel
d < 0 use lower pixel

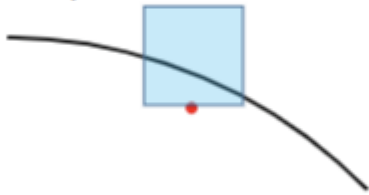
- Given x_1, y_1, x_2, y_2
- Calculate $\Delta x, \Delta y$
- $d_0 = 2\Delta y - \Delta x$
- SetPixel(x_0, y_0)
- At each x_k , k starts from k=0, do $\Delta x - 1$ steps
 - If $d_k > 0$, SetPixel($x_k + 1, y_k$) and $d_{k+1} = d_k + 2\Delta y - 2\Delta x$
 - If $d_k < 0$, SetPixel($x_k + 1, y_k + 1$) and $d_{k+1} = d_k + 2\Delta y$

Bresenham扩展算法: Mid-point algorithm

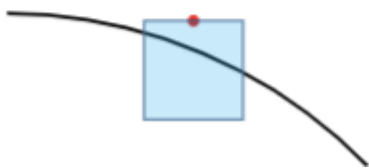
Midpoint Circle Algorithm

- Decision Function:

- $d_k = D(x_k + 1, y_k - 0.5) = (x_k + 1)^2 + (y_k - 0.5)^2 - r^2$
- If $d_k < 0$, the mid point is inside the circle, choose upper



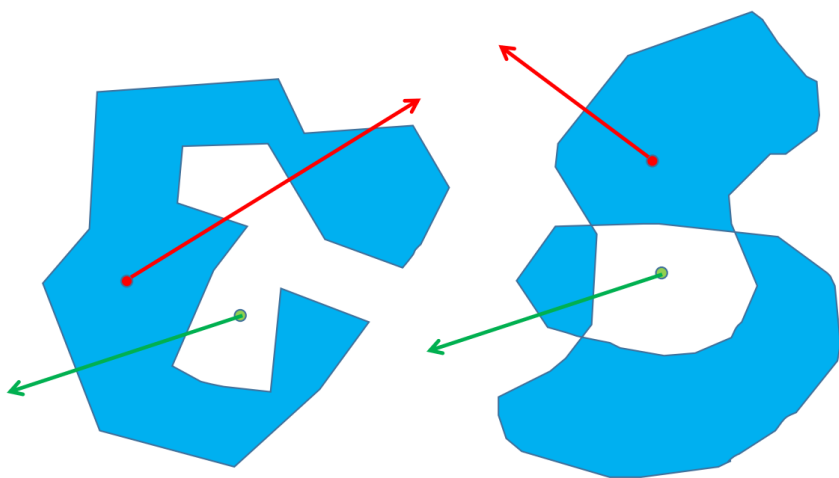
- If $d_k > 0$, the mid point is outside the circle, choose lower
- If $d_k = 0$, the mid point is on the circle, choose lower



判断点在图形内外: **Odd-Even rule**

Inside polygon rule

- Odd-Even rule



着色方法: **Flood Fill**

Triangulation 三角化

Triangulation

- Rasterization algorithms can take advantage of triangle properties
- Graphics hardware is optimized for triangles
- Because triangle drawing is so fast, many systems will subdivide polygons into triangles prior to scan conversion

4 Illumination Model and Shading

HSV color model

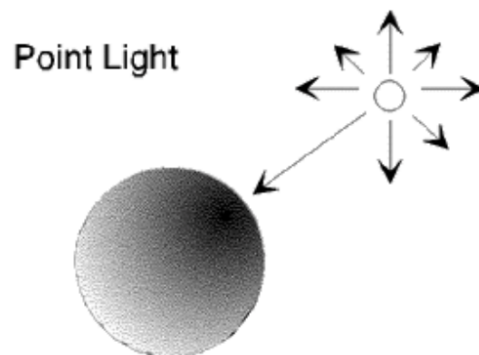
- **H**ue, **S**aturation, and **V**alue (Brightness)
- Rearrange the geometry of RGB in an attempt to be more intuitive and perceptually relevant

Global Illumination (全局照明)

1. Point Light

$$f_{radatten} = \frac{1}{a_0 + a_1 d + a_2 d^2}$$

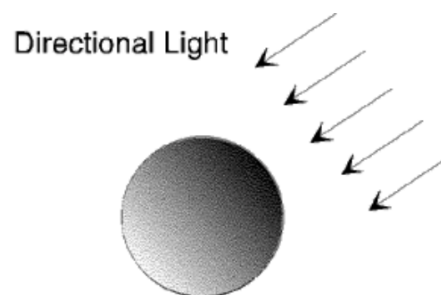
$$I_d = f_{radatten} I$$



2. Directional light

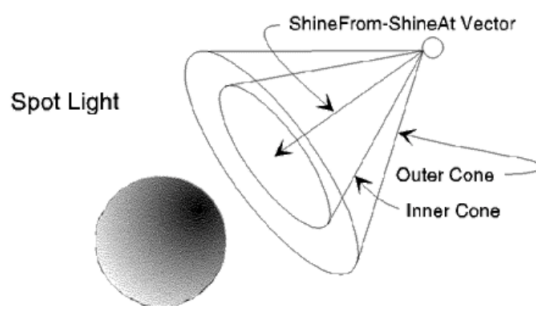
Directional light

- This type can be imagined as a point light source lying in infinity, e.g. Sun light
- Light rays from such a source are radiated in parallel.
- Attributes to specify a directional light source
 - Direction V (v_x, v_y, v_z)
 - Intensity I
 - No attenuation Why?

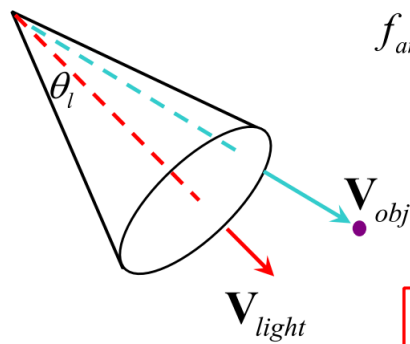


3. Spotlight

Spotlight



- Besides position and color
- the apex angle of the lighting cone needs to be specified
- (px, py, pz) , I , (tx, ty, tz) and Theta



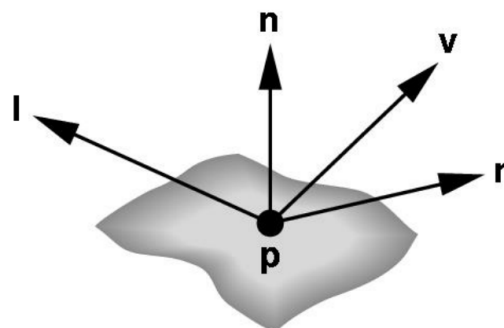
$$f_{angatten} = \begin{cases} 0.0, & \text{when } \mathbf{V}_{obj} \cdot \mathbf{V}_{light} < \cos \theta_l \\ \left(\mathbf{V}_{obj} \cdot \mathbf{V}_{light} \right)^\alpha, & \text{otherwise} \end{cases}$$

where α is a constant

$f_{radatten}, f_{angatten}$ can be combined

Phong reflection model

- I = vector to light source
- n = surface normal
- v = vector to viewer
- r = reflection of I at p (determined by I and n)



- Light components
 - Ambient I_a , Diffuse I_d and Specular I_s
- Material coefficients for each light component
 - k_a , k_d , k_s and α
- Therefore:
- $I = L_a k_a + L_d k_d \max(I \cdot n, 0) + L_s k_s \max(r \cdot v)^\alpha, 0)$

- Attenuation

- $I_{\text{atten}} = I_a + \frac{1}{a_0 + a_1 d + a_2 d^2} (I_d + I_s)$

- How about If there are multiple lights?

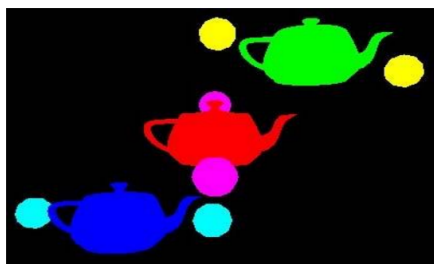
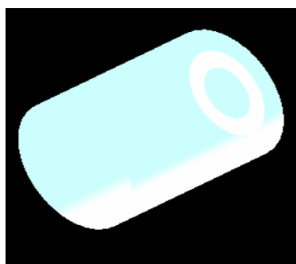
- $I = L_a k_a + \sum_{i=1}^m (L_{d,i} k_d \max(I \cdot n, 0) + L_{s,i} k_s \max((r_i \cdot v)^\alpha, 0))$

- How about shadows?

- $I = L_a k_a + s_i \sum_{i=1}^m (L_{d,i} k_d \max(I \cdot n, 0) + L_{s,i} k_s \max((r_i \cdot v)^\alpha, 0))$

1. Ambient reflection (环境反射)

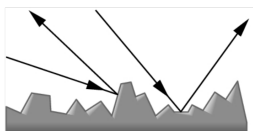
- Ambient reflection is a constant for a scene L_a
- Different surface can have different ambient reflection coefficient k_a ($0 \leq k_a \leq 1$)
- So, if only consider ambient lighting, the illumination at a point simply is $I_a = L_a * k_a$



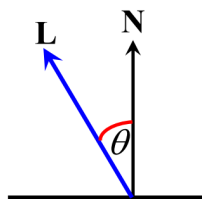
Rendering results when only ambient lighting is considered

2. Diffuse reflection (漫反射)

- Diffuse reflector scatters light
- Assume equally all direction
- Called Lambertian surface
- Diffuse reflection coefficient k_d , $0 \leq k_d \leq 1$
- Angle of incoming light still critical

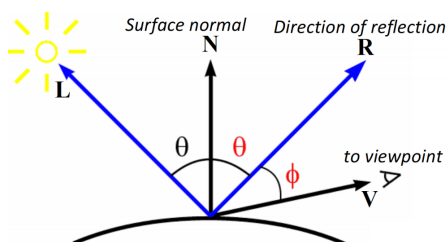


- l = unit vector to light
- n = unit surface normal
- θ = angle to normal
- $\cos\theta = l \cdot n$
- $I_d = k_d (l \cdot n) L_d$
- If $\theta > 90^\circ$ ($l \cdot n < 0$), $I_d = 0$

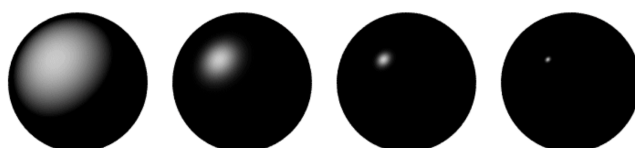
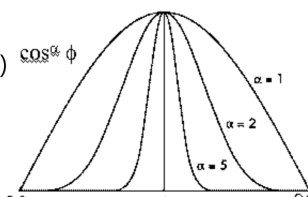


3. Specular reflection (镜面反射)

- $I_s = \max(k_s L_s \cos^\alpha \phi, 0.0)$
- $\cos\phi = r \cdot v$
- What is α ?



- Higher α is narrower
- $I_s = \max(k_s L_s \cos^\alpha \phi, 0.0)$



Blinn-Phong reflection model

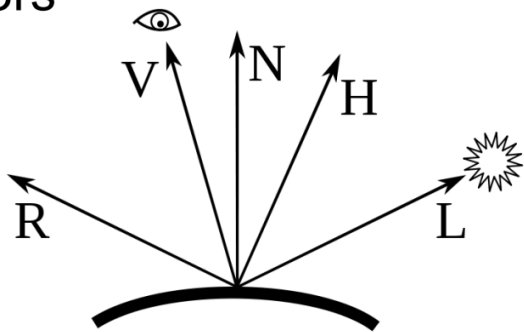
用半程向量（Half-Vector）代替反射向量，好处：

- 1. 计算量更小
- 2. 更符合现实光照特性

- In Phong model, one must continually recalculate the dot product of r and v. Instead, one calculates a halfway vector between the viewer and light-source vectors

$$\mathbf{H} = \frac{\mathbf{L} + \mathbf{V}}{\|\mathbf{L} + \mathbf{V}\|}$$

- $L_s k_s (r \cdot v)^\alpha \rightarrow L_s k_s (n \cdot h)^\alpha$



Surface Shading Method

1. Flat shading

Flat shading

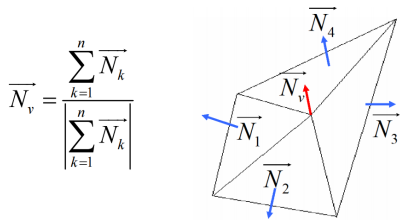
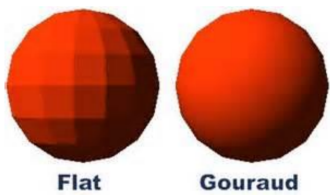
- The simplest one, also called as “constant intensity surface rendering”
- One illumination calculation per polygon
- Assign all pixels inside each polygon the same color, therefore reveal polygons and fast
- OK for polyhedral objects, Not good for smooth surfaces



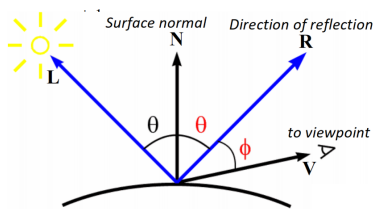
2. Gouraud Shading

以临街面计算点法向量→计算改点的Lighting→根据点的颜色内插其他像素颜色

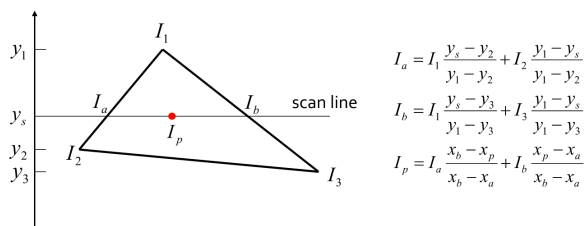
- Produce continuous shading of surfaces represented by polygon meshes
- An interpolation method
- Step 1: Normal averaging
 - estimate the normal of each vertex by averaging the surface normals of the polygons that meet at each vertex



- Step 2: Vertex Lighting
 - compute the color for each vertex based on a shading model
 - $I = L_a k_a + \sum_{i=1}^m (L_{d,i} k_d \max(\mathbf{l} \cdot \mathbf{n}, 0) + L_{s,i} k_s (\mathbf{h}_i \cdot \mathbf{n})^\alpha)$



- Step3: Interpolation
 - for each **screen pixel** that represents the polygonal mesh, color is interpolated from the color values calculated at the vertices
 - Bilinear interpolation or barycentric coordinates



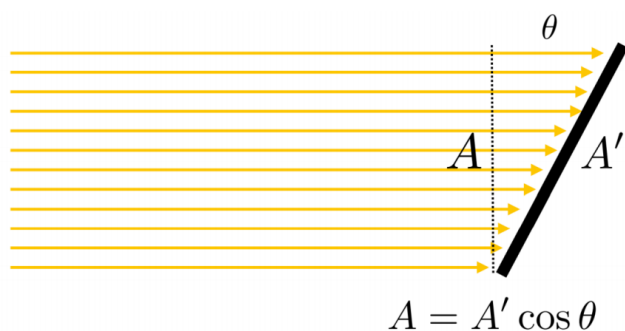
3. Phong shading

- Different from Gouraud shading, Phong shading interpolate normals rather than colors
- It interpolates surface normals across **rasterized polygons** and computes pixel colors based on the interpolated normals and a reflection model

How to implement shading in OpenGL? 见PPT8末

Physically-based Rendering

Radiant\Radiant flux (辐射通量, 每秒能量大小) \Irradiance (辐射度, 每单位能量大小)



A = projected area of surface relative to direction of beam

$$E = \frac{\Phi}{A'} = \frac{\Phi \cos \theta}{A} \quad \text{Lambert's Law!}$$

Rendering equation 渲染方程

渲染方程(rendering equation)是用于计算场景中物体表面点的出射光的数学公式

$$L_o(\mathbf{p}, \omega_o) = L_e(\mathbf{p}, \omega_o) + \int_{\mathcal{H}^2} f_r(\mathbf{p}, \omega_i \rightarrow \omega_o) L_i(\mathbf{p}, \omega_i) \cos \theta d\omega_i$$

Diagram annotations for the rendering equation:

- $L_o(\mathbf{p}, \omega_o)$: outgoing/observed radiance
- $L_e(\mathbf{p}, \omega_o)$: emitted radiance (e.g., light source)
- $\mathbf{p}$: point of interest
- ω_o : direction of interest
- $\int_{\mathcal{H}^2}$: all directions in hemisphere
- $f_r(\mathbf{p}, \omega_i \rightarrow \omega_o)$: scattering function
- $L_i(\mathbf{p}, \omega_i)$: incoming radiance
- ω_i : incoming direction
- $\cos \theta$: angle between incoming direction and normal

其中:

$L_o(\mathbf{p}, \omega_o)$ 是从点 $\mathbf{p}$ 向方向 ω_o 观察到的出射光, 即观察到的辐射光。

$L_e(\mathbf{p}, \omega_o)$ 是从点 $\mathbf{p}$ 向方向 ω_o 发射的光, 发光的物体有这一项。

$f_r(\mathbf{p}, \omega_i \rightarrow \omega_o)$ 是双向反射分布函数 (BRDF), 描述了从方向 ω_i 入射到点 $\mathbf{p}$ 的光如何被反射到方向 ω_o 。

$L_i(\mathbf{p}, \omega_i)$ 是从方向 ω_i 入射到点 $\mathbf{p}$ 的入射光。

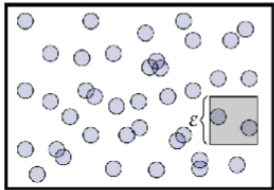
θ 是入射光方向与表面法线之间的角度。

$d\omega_i$ 是立体角, 表示从方向 ω_i 来的光的一小部分。

$\mathcal{H}^2$ 表示整个半球空间。

Irradiance

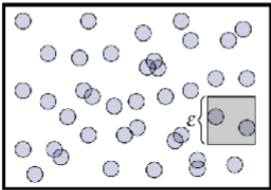
- More specific than an arbitrary area, Density
- #hits per second, per unit area
- $E = d\Phi/dA$



Estimate of “radiant flux density”: $2/\epsilon^2$

辐射

- 比任意面积更具体，密度 · 每秒 #hits，每单位面积
- $E = d\Phi/dA$



“辐射通量密度” 估计: $2/\epsilon \ 2/\epsilon^2$

辐照度（Irradiance）是指单位面积上接收到的辐射通量，即单位时间内通过单位面积的光能量。

Radiance

- Radiance is the solid angle density of irradiance

$$L(p, \omega) = \lim_{\Delta \omega \rightarrow 0} \frac{\Delta E_\omega(p)}{\Delta \omega}$$

- where E_ω denotes that the differential surface area is oriented to face in the direction ω



Energy per unit time per unit area per unit solid angle...!

光芒

- 辐射度（Radiance）是辐照度的立体角密度

$$L(p, \omega) = \lim_{\Delta \omega \rightarrow 0} \frac{\Delta E_\omega(p)}{\Delta \omega}$$

- 其中 E_ω 表示差分表面积朝向 Ω 方向的面



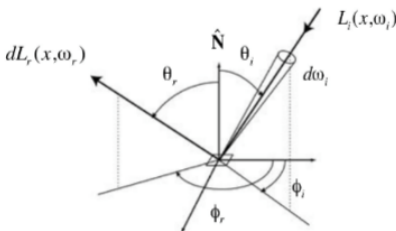
能量 每单位时间 每单位面积 每单位立体角...!

辐射率（Radiance）是辐照度的立体角密度，用于衡量在特定方向上单位时间、单位面积、单位立体角内传播的光能量。

BRDF（Bidirectional Reflectance Distribution Function）

物体定义的在不同反射方向上的反射率，通俗点说就是人眼（也就是反射光线）在不同的方向看到物体某一点不同的像素值。

Radiometric description of BRDF

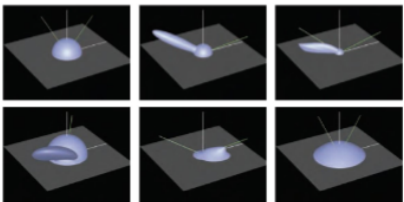


$$f_r(\omega_i \rightarrow \omega_o) = \frac{dL_o(\omega_o)}{dE_i(\omega_i)} = \frac{dL_o(\omega_o)}{dL_i(\omega_i) \cos \theta_i}$$

“For a given change in the incident irradiance, how much does the exitant radiance change?”

Reflecting off a surface: the BRDF

- “Bidirectional reflectance distribution function”
- Encodes behavior of light that “bounces off” surface
- Given incoming direction ω_i , how much light gets scattered in any given outgoing direction ω_o ?
- Describe as distribution $f_r(\omega_i \rightarrow \omega_o)$

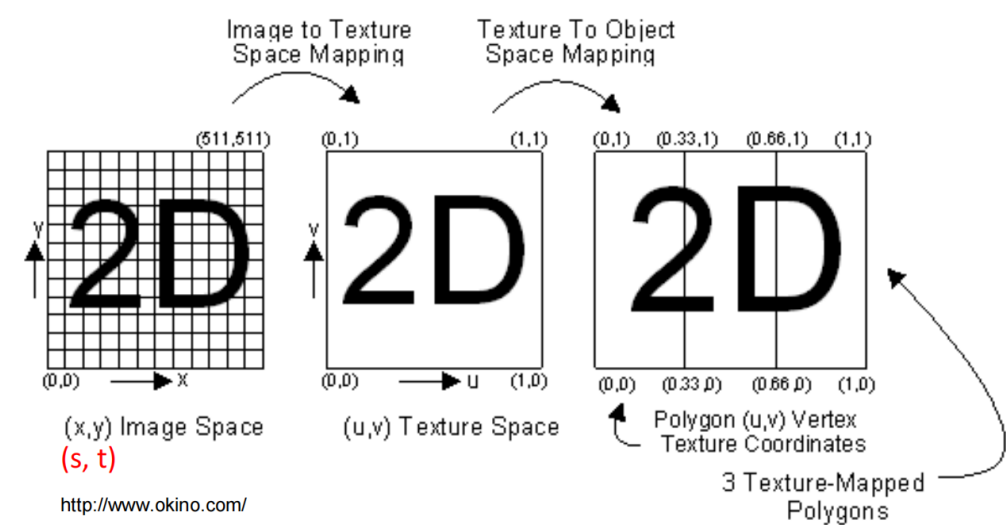


Texture Mapping

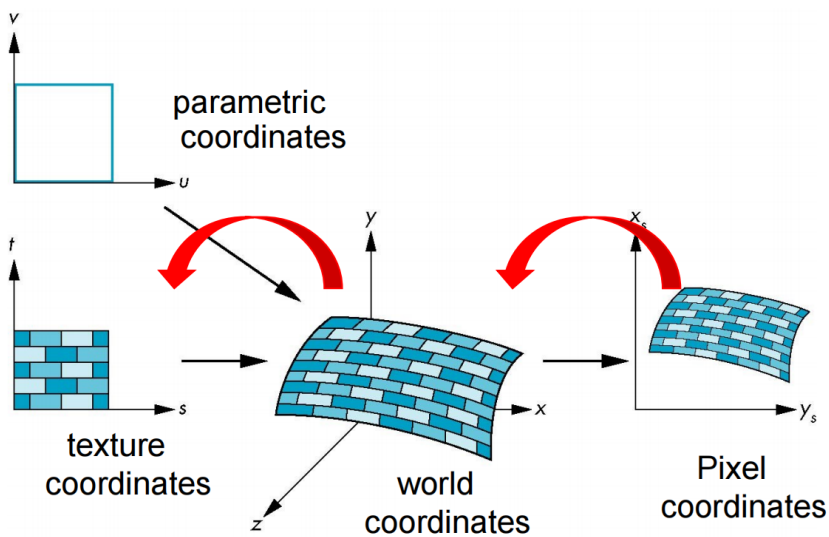
- An “image” is pasted onto the 3D model
- The “image” is called a **texture map**, the pixels of the image are referred as **texels** and referenced using **UV** coordinates

- Material is the intrinsic properties of a surface defining how the lighting interacts with the surface
 - per vertex (or per polygon)
- Texture is an “image-based” data describing appearance property
 - per texel/pixel

UV Coordinate



Backward Mapping（反向映射）



- **反向映射**：从屏幕空间回溯到纹理空间，找到像素对应的纹理坐标，然后采样纹理值。
 - 例如：给定屏幕上的一个像素 (x, y) ，通过变换找到纹理坐标 (u, v) ，从而读取纹理的颜色信息。
- 与之相对的是**正向映射（forward mapping）**，它从纹理空间直接将纹理数据投影到屏幕空间。

优点：

1. 避免空洞问题

反向映射确保屏幕空间的每个像素都有对应的纹理采样点，不会出现未覆盖区域。

2. 更精确的纹理采样

反向映射可以精确控制每个屏幕像素的纹理坐标，并使用插值方法（如双线性插值或各向异性过滤）来提升纹理的质量。

3. 计算效率更高

4. 适配任意分辨率

[图片]



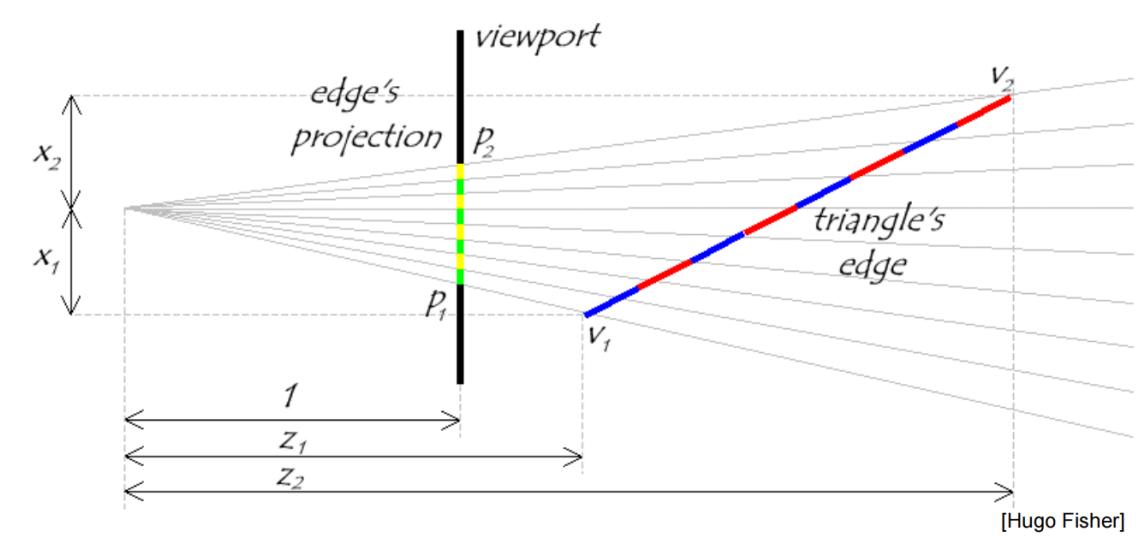
Hasky 12月24日 22:48
不懂，求教



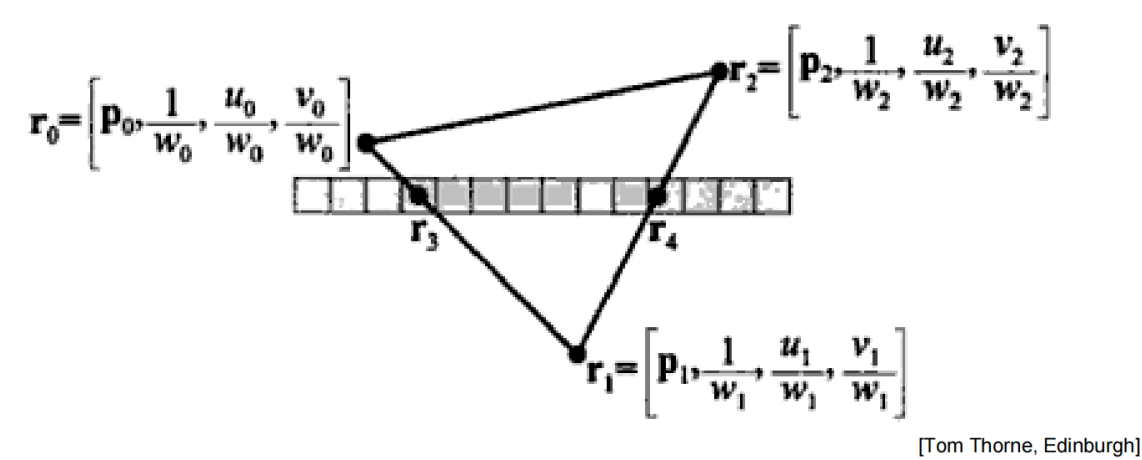
赵嘉禾 12月25日 22:38
应该是这样，首先这里用的应该是反向映射，也就是拿着 xy 去纹理坐标系找对应的 uv。但因为有了近大远小，所以每个像素对应 uv 就会出现问题

需要透视校正

- Uniform steps on the image plane does not correspond to uniform steps along the edge

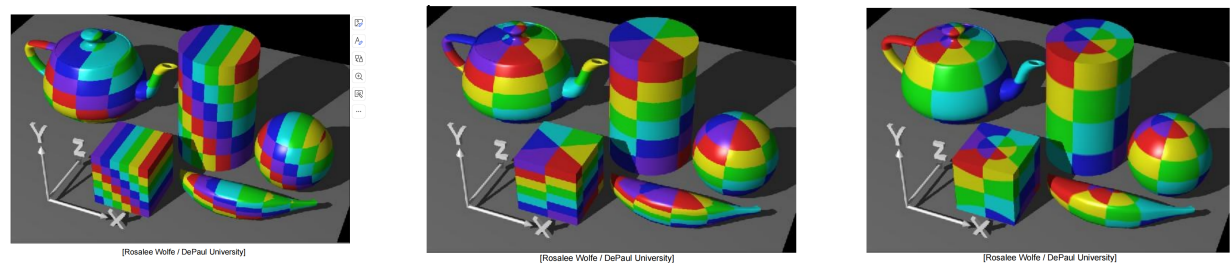


- (u, v) should be divided by the depth
- using (u/w, v/w) in homogeneous coordinates



How to generate texture coordinates

Orthogonal mapping、Cylindrical mapping、Spherical mapping



远处纹理处理：Mipmapping

- When several texels covering a pixel
- or the textured surface is magnificated too much (we see pixels!)
- Use a texture of multiple resolutions
- Switch the resolution according to the number of texels in one pixel

Other texturing techniques



赵嘉禾 12月25日 23:42

不同 mapping 区别

3 Rendering

asterize an arbitrary poly
ping vs Bump-mapping vs
ment mapping vs Cubic m
ble usages of texture obje
map vs Shadow volume?



赵嘉禾 12月25日 23:42

谁来回答一下

Other texturing techniques

- Mipmapping：Texture of multiple resolutions.
- Bump mapping：Use texture to disturb the normal.
- Displacement mapping：Use the texture map to actually move the surface point.
- Environmental mapping：Simulate the reflection using texture mapping.
- Cubic mapping: eg. Sky box.

5 Mesh

Manifold（流形）

Mesh validity

- manifold
 - no edge is shared by more than two polygons; the faces adjacent to a vertex form a single ring
 - edge points: each edge must have exactly 2 triangles
 - vertex points: each vertex must have one loop of triangles
- manifold with boundary
 - weaken rules to allow boundaries
 - The faces adjacent to a vertex form an incomplete ring

[Steve Marschner 2013]

网格有效性

- 流形
 - 两个以上的多边形均不共享任何边；与顶点相邻的面形成一个环
 - Edge Points: 每条边必须恰好有 2 个三角形
 - Vertex Points: 每个顶点必须有一个三角形循环
- 带边界的歧管
 - 弱化规则以允许边界
 - 与顶点相邻的面形成不完整的环

[Steve Marschner 2013]

Topological validity

- Consistent orientation
 - Which side is the “front” or “outside” of the surface and which is the “back” or “inside?”
 - rule: you are on the outside when you see the vertices in counter-clockwise order
 - in mesh, neighboring triangles should agree about which side is the front!
 - caution: not always possible

[Steve Marschner 2013]

拓扑有效性

- 一致的方向
 - 哪一侧是表面的“正面”或“外部”，哪一侧是“背面”或“内部”？
 - 规则：当您按逆时针顺序查看顶点时，您位于外部
 - 在网格中，相邻的三角形应该就哪一侧是正面达成一致！
- 注意：并非总是可行

[史蒂夫·马施纳]

Geometric Validity

- generally want non-self-intersecting surface
- hard to guarantee in general
 - because far-apart parts of mesh might intersect

[Steve Marschner 2013]

几何有效性

- 一般需要非自相交表面 · 一般难以保证
- 因为网格中相距较远的部分可能会相交

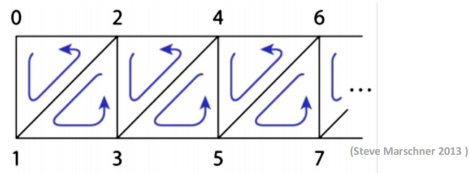
[史蒂夫·马施纳]

Polygon 记录的几种方法

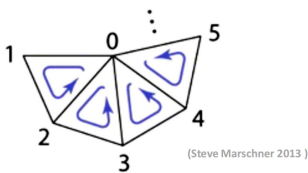
- *.obj Wavefront obj
 - #vertices
 - v x, y, z
 - ...
 - #faces
 - f i1, i2, i3, ..., in
- Note there can be more information such as normal, texture etc.
 - #normal
 - vn nx, ny, nz
 - vt u, v, w
 - ...

Triangle strips/Triangle fans

- e. g., 0, 1, 2, 3, 4, 5, 6, 7, ... leads to (0 1 2), (2 1 3), (2 3 4), (4 3 5), (4 5 6), (6 5 7), ...
- for long strips, this requires about one index per triangle



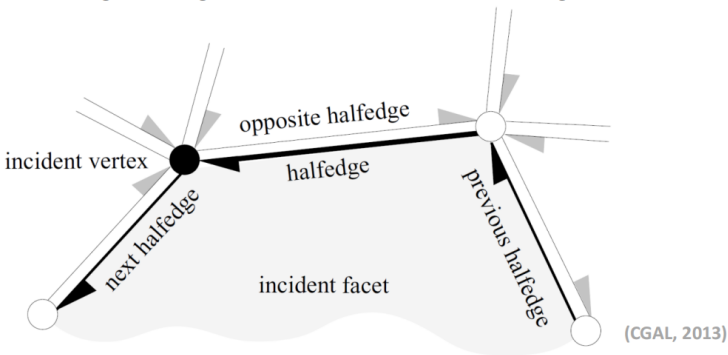
- e. g., 0, 1, 2, 3, 4, 5, ... leads to (0 1 2), (0 2 3), (0 3 4), (0 3 5), ...
- Memory considerations exactly the same as triangle strip



Half-edge Model

Half-edge data structure

- A half-edge data structure is an **edge-centered** data structure capable of maintaining incidence information of vertices, edges and faces
- Instead of a single edge, 2 **oriented** “half edges”



!!! 记住数据结构

Half Edge Table

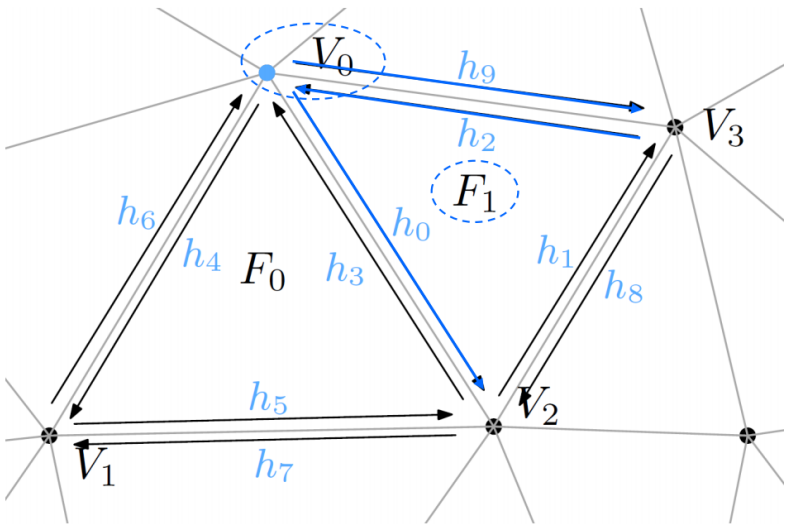
$h_0: V_2; F_1; h_1; h_3$
 $h_2: V_0; F_1; h_0; h_9$
 $h_3: V_0; F_0; h_4; h_0$

Face Table

$F_0: h_3$
 $F_1: h_2$

Vertex Table

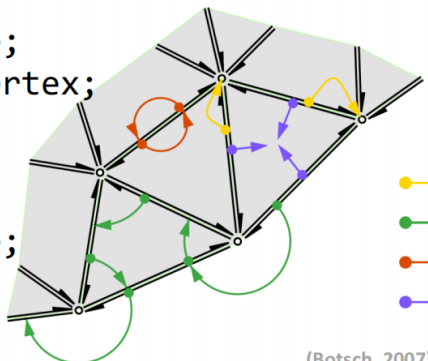
$V_0: h_2$
 $V_1: h_4$



```
struct Halfedge {
    Halfedge* next;
    Halfedge* opposite;
    Face* incident_face;
    Vertex* incident_vertex;
}

struct Face{
    Halfedge * halfedge;
}

struct Vertex{
    Halfedge* incident_halfedge;
}
```



(Botsch, 2007)

查询方法：

3、邻接查询

大多数邻接查询的答案直接存储在边、顶点和面的数据结构中。例如，与半边相邻的顶点或面可以很容易地找到：

```
1 HE_vert* vert1 = edge->vert;
2 HE_vert* vert2 = edge->pair->vert;
3
4 HE_face* face1 = edge->face;
5 HE_face* face2 = edge->pair->face;
```

一个稍微复杂一些的例子是迭代与面相邻的半边。由于面周围的半边形成圆形链表，并且面结构存储了指向这些半边之一的指针，我们这样做：

```
1 HE_edge* edge = face->edge;
2
3 do {
4
5     // do something with edge
6     edge = edge->next;
7
8 } while (edge != face->edge);
```

同样，我们可能有兴趣迭代与特定顶点相邻的边或面。回到前面的图，你可能会看到，除了围绕面形成圆形链表之外，半边还在顶点周围形成循环。迭代过程和查询与顶点相邻的边或面一样，这是在 C 中的实现：

```
1 HE_edge* edge = vert->edge;
2
3 do {
4
5     // do something with edge, edge->pair or edge->face
6     edge = edge->pair->next;
7
8 } while (edge != vert->edge);
```

Vertex Removal、Face Collapse、Edge Collapse

Limitations of Polygonal Mesh:

- 1. Planar facets
- 2. Fixed resolution
- 3. Deformation is difficult
- 4. Hard for parameterization

Spline Curve 样条曲线

Hermite spline curve

$$f(t) = at^3 + bt^2 + ct + d$$

$$[p_0 \ p_1 \ p_2 \ p_3] \begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix} \begin{bmatrix} t^3 \\ t^2 \\ t \\ 1 \end{bmatrix}$$

$$Q(t) = GBT(t) = \text{Geometry } G \cdot \text{Spline Basis } B \cdot \text{Power Basis } T(t)$$

在曲线和曲面建模中，**GBT** 是一个常见的术语，它代表了 **Geometry**（几何体）、**Spline Basis**（样条基）和 **Power Basis**（幂基）。这些概念通常用于定义和计算曲线的数学表示。它们分别涉及不同的曲线表示方法和基函数。

Continuity



- C^0 continuous
 - curve/surface has no breaks/gaps/holes
 - "watertight"
- C^1 continuous
 - curve/surface derivative is continuous
 - "looks smooth, no facets"
- C^2 continuous
 - curve/surface 2nd derivative is continuous
 - Actually important for shading

连续性

- C^0 连续
 - 曲线 / 曲面没有破断 / 间隙 / 孔
 - “水密” C^1 连续
- C^1 连续
 - 曲线 / 曲面导数是连续的
 - “看起来光滑，没有刻面” C^2 连续
- C^2 连续
 - 曲线 / 曲面 2nd 导数是连续的
 - 实际上对阴影很重要

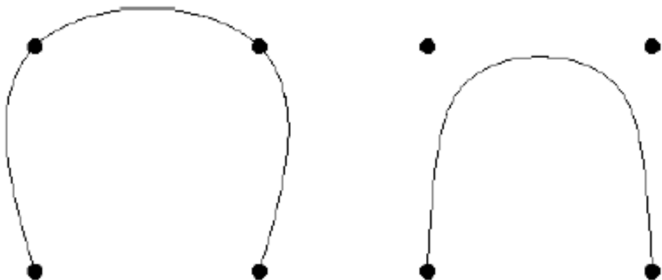
Subdivision vs Splines

- In regular regions, behavior is identical
- At extraordinary vertices, achieve $C1$
 - near extraordinary, different from splines
- Linear everywhere
 - mapping from parameter space to 3D is a linear combination of the control points

细分 vs 样条线

- 在常规区域中，行为是相同的
- 在非平凡顶点处，实现 $C1$
 - 近乎非凡，与样条曲线不同
- 无处不在的线性
 - 从参数空间到 3D 的映射是控制点的线性组合

- Smooth curve defined by some control points
- Moving the control points changes the curve
- **Interpolation vs approximation**



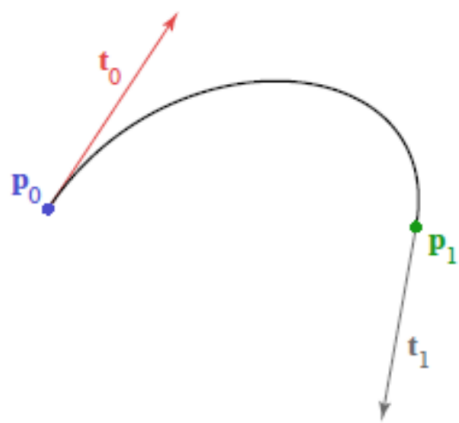
Interpolation Spline:

- Given points $P_0, P_1, \dots, P_n$, find lowest degree polynomial which passes through the points

$$x(t) = a_{n-1}t^{n-1} + \dots + a_2t^2 + a_1t + a_0$$
$$y(t) = b_{n-1}t^{n-1} + \dots + b_2t^2 + b_1t + b_0$$

Hermite Spline:

- Constraints: endpoints and tangents



$$x(t) = at^3 + bt^2 + ct + d$$
$$x'(t) = 3at^2 + 2bt + c$$
$$x(0) = x_0 = d$$
$$x(1) = x_1 = a + b + c + d$$
$$x'(0) = x'_0 = c$$
$$x'(1) = x'_1 = 3a + 2b + c$$

$$d = x_0$$
$$c = x'_0$$
$$a = 2x_0 - 2x_1 + x'_0 + x'_1$$
$$b = -3x_0 + 3x_1 - 2x'_0 - x'_1$$

参数t：表示曲线上点的相对坐标，取0~1

已知[p0,p1,x0,x1]，分别指曲线头与尾的位置（p）和切线（t）

$$f(t) = at^3 + bt^2 + ct + d$$

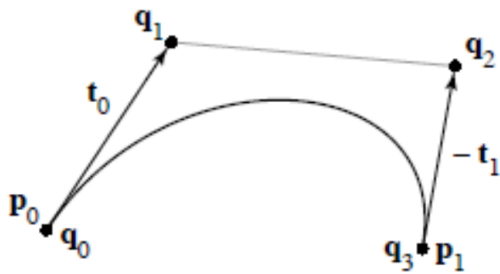
$$[p_0 \ p_1 \ t_0 \ t_1] \begin{bmatrix} 2 & -3 & 0 & 1 \\ -2 & 3 & 0 & 0 \\ 1 & -2 & 1 & 0 \\ 1 & -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} t^3 \\ t^2 \\ t \\ 1 \end{bmatrix}$$

$$f(t) = b_0(t)p_0 + b_1(t)p_1 + b_2(t)p_2 + b_3(t)p_3$$

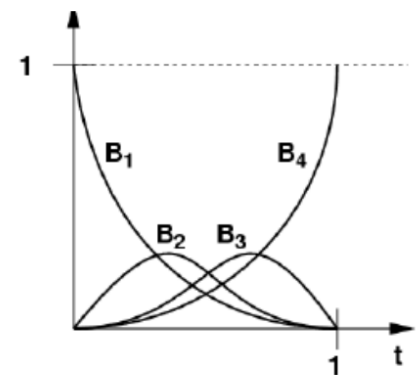
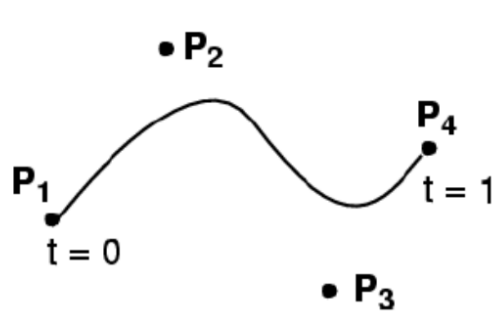
Bezier Spline:

Hermite to Bézier

$$p_0 = q_0$$
$$p_1 = q_3$$
$$t_0 = 3(q_1 - q_0)$$
$$t_1 = 3(q_3 - q_2)$$

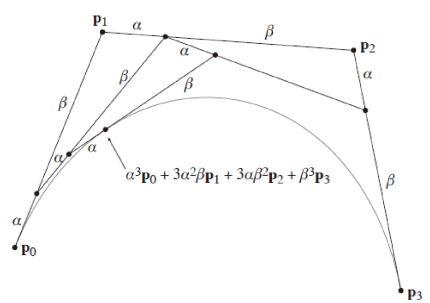


$$\bullet [q_0 \ q_1 \ q_2 \ q_3] \begin{bmatrix} 1 & 0 & -3 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & -3 \\ 0 & 1 & 0 & 3 \end{bmatrix} \begin{bmatrix} 2 & -3 & 0 & 1 \\ -2 & 3 & 0 & 0 \\ 1 & -2 & 1 & 0 \\ 1 & -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} t^3 \\ t^2 \\ t \\ 1 \end{bmatrix}$$



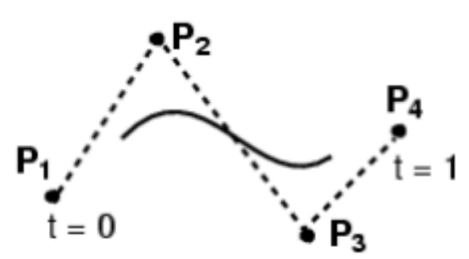
$$Q(t) = (1 - t)^3 P_1 + 3t(1 - t)^2 P_2 + 3t^2(1 - t) P_3 + t^3 P_4$$

$$\begin{aligned} p_{3,0} &= \alpha p_{2,0} + \beta p_{2,1} \\ &= \alpha \alpha \alpha p_0 + \alpha \alpha \beta p_1 + \alpha \beta \alpha p_1 + \alpha \beta \beta p_2 \\ &\quad \beta \alpha \alpha p_1 + \beta \alpha \beta p_2 + \beta \beta \alpha p_2 + \beta \beta \beta p_3 \\ &= \alpha^3 p_0 + 3\alpha^2 \beta p_1 + 3\alpha \beta^2 p_2 + \beta^3 p_3 \end{aligned}$$



BSpline:

$$B_{B-Spline} = \frac{1}{6} \begin{pmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 0 & 3 & 0 \\ 1 & 4 & 1 & 0 \end{pmatrix}$$



BSplines

$$Q(t) = \frac{(1-t)^3}{6} P_{i-3} + \frac{3t^2 - 6t^2 + 4}{6} P_{i-2} + \frac{-3t^2 + 3t^2 + 3t + 1}{6} P_{i-1} + \frac{t^3}{6} P_i$$

$$Q(t) = \mathbf{GBT}(t) \quad B_{B-Spline} = \frac{1}{6} \begin{pmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 0 & 3 & 0 \\ 1 & 4 & 1 & 0 \end{pmatrix}$$

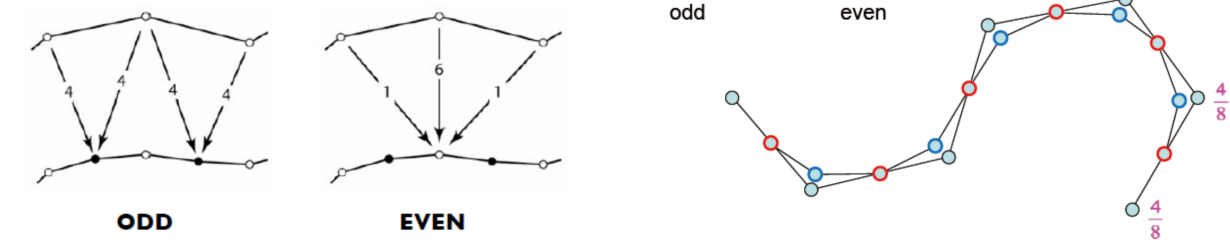
NURBS是非均匀有理B样条 (Non-Uniform Rational B-Splines)

- **non-uniform** = different spacing between the blending functions,
- **rational** = quotients of polynomials

Subdivision curves

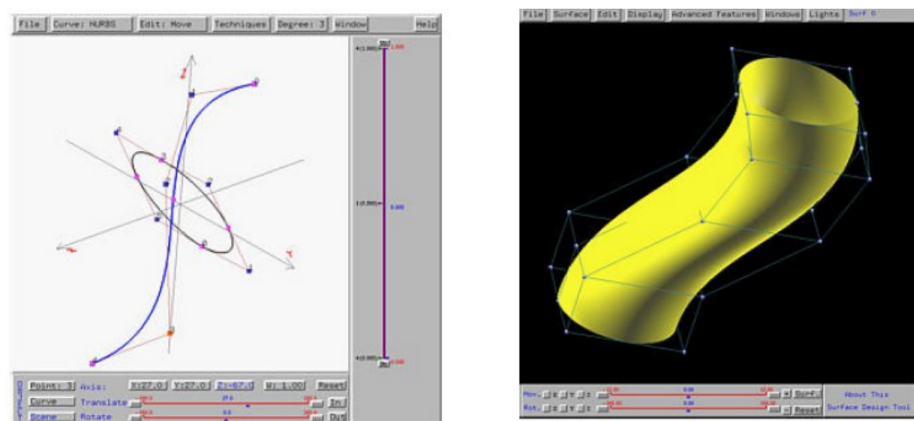
Subdivision for B-splines:

- Control vertices of refined spline are linear combinations of the coarse spline



From curves to surfaces

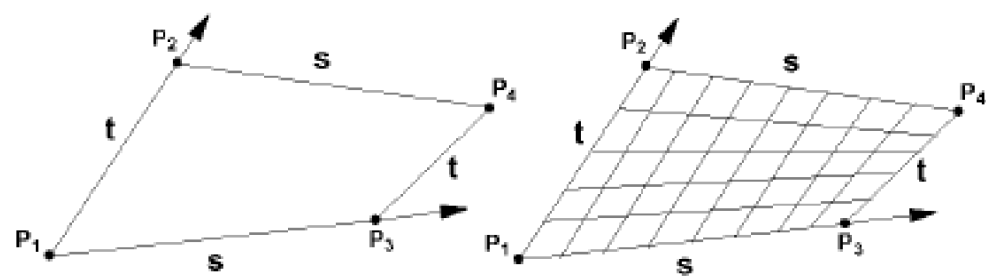
Sweeping



Bilinear Surface Patch（二线性曲面片）

用几个点控制面

- $L(P_1, P_2, \alpha) = (1 - \alpha)P_1 + \alpha P_2$
- $Q(s, t) = L(L(P_1, P_2, t), L(P_3, P_4, t), s)$



Subdivision surfaces

Two parts to subdivision process :

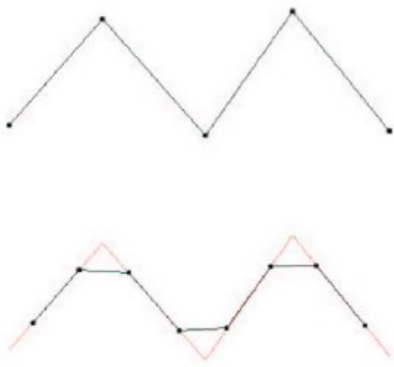
- Subdividing the mesh (computing new topology)
- Positioning the vertices (computing new geometry)

For surfaces: two kinds of rules (still called odd and even)

New vertex’ s position is a weighted average of positions of old vertices that are nearby in the mesh

Chaikin’ s Algorithm

- 1 quarter 3 quarter algorithm



Shadow

Ground shadow、Shadow texture、Shadow map、Shadow volume、Soft shadow

6 Ray tracing

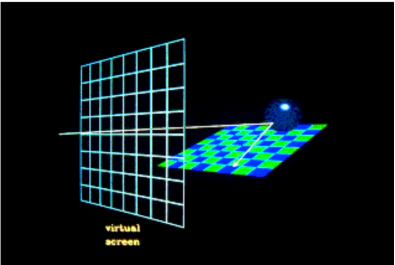
- OpenGL is based on a pipeline model in which primitives are rendered one at time
 - No shadows (shadow maps etc.)
 - No multiple reflections (environment maps etc.)
- Global approaches
 - Rendering equation
 - Ray tracing
 - Radiosity
 - Easily making effects: shadows, reflections, transparency

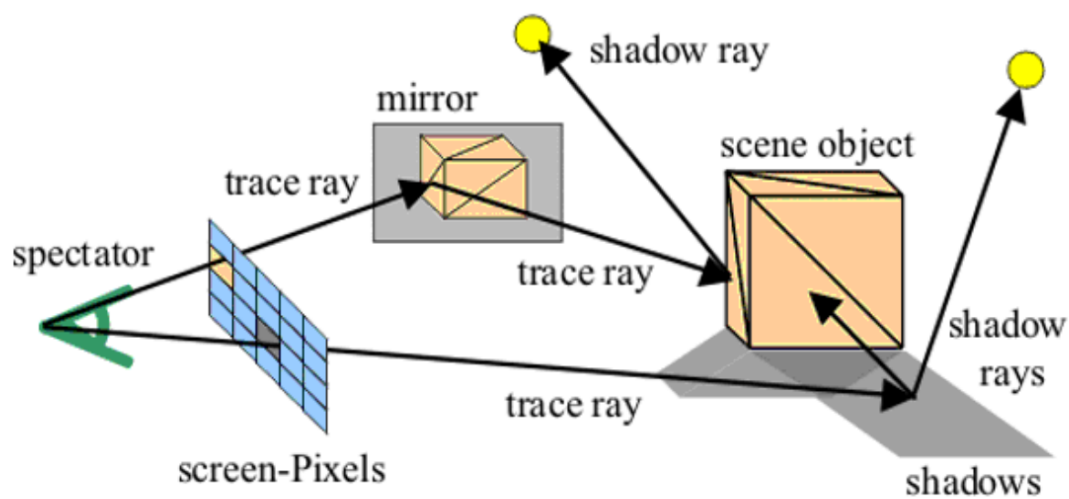
特性	光栅化	光线追踪
优势	实时渲染速度快，支持大规模场景渲染	物理准确的光照、阴影、反射和折射效果
劣势	对复杂光照效果的模拟不如光线追踪准确	高计算成本，实时性较差

- If the ray hits an object, we want to know if that point on the object is in a shadow.
- So, when the ray hits an object, a secondary ray, called a **"shadow" ray**, is shot towards the light sources.

- If this shadow ray hits another object before it hits a light source, then the first intersection point is in the shadow of the second object.
- For a simple illumination model this means that we only apply the ambient term for that light source.

- Also, when a ray hits an object, a **reflected ray** is generated which is tested against all of the objects in the scene.





Ray Casting (光线追踪)

Ray Casting (光线投射)

- 是一种基础的渲染技术，用于确定视点与场景中物体之间的可见性。
- 将从摄像机视点出发，通过屏幕像素投射射线，计算射线与场景物体的第一个相交点，确定像素的颜色。
- **不考虑光的复杂行为**（如反射、折射或全局光照）。
- 常用于基本的可见性计算和快速渲染。

Ray Tracing (光线追踪)

- 在光线投射的基础上，进一步模拟光线在场景中的传播和交互。
- 不仅计算光线与物体的初次交点，还递归地追踪光线的反射、折射和阴影。
- 能够产生更加逼真的渲染效果（包括反射、折射、阴影和全局光照）。

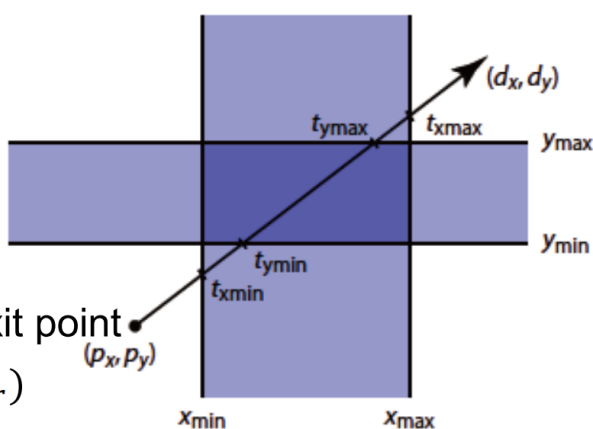
Ray-box Intersection (不算太懂，box是什么?)

• Find the enter point and exit point

- $t_{xenter} = \min(t_{xmin}, t_{xmax})$
- $t_{yenter} = \min(t_{ymin}, t_{ymax})$
- $t_{xexit} = \max(t_{xmin}, t_{xmax})$
- $t_{yexit} = \max(t_{ymin}, t_{ymax})$

• Last enter point and first exit point

- $t_{enter} = \max(t_{xenter}, t_{yenter})$
- $t_{exit} = \min(t_{xexit}, t_{yexit})$

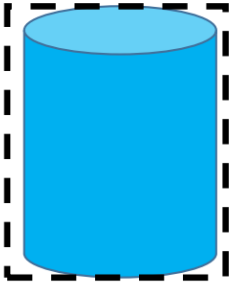
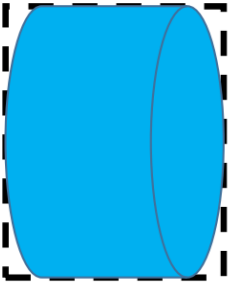
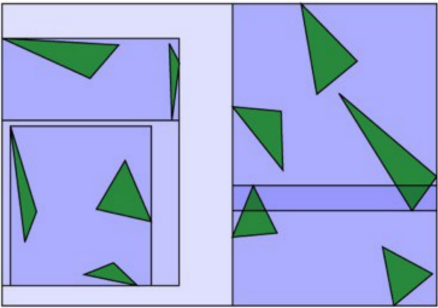


To accelerate:

Try to avoid meaningless intersections by using auxiliary data structures

1. Bounding volume hierarchy (BVH)

- A tree



2. Octree

7 OpenGL

```
// ...: 初始化代码（只运行一次（除非你的物体频繁改变）） :: ..
// 1. 绑定VAO
glBindVertexArray(VAO);
// 2. 把顶点数组复制到缓冲中供OpenGL使用
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
// 3. 设置顶点属性指针
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);

[...]

// ...: 绘制代码（渲染循环中） :: ..
// 4. 绘制物体
glUseProgram(shaderProgram);
glBindVertexArray(VAO);
someOpenGLFunctionThatDrawsOurTriangle();
```